

SOLIDWORKS API (NET VBA C#) — Examples (13/50):

Create_Macro_Feature_Subfeature_Example_VB

Create_Planar_Surface_Feature_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Create Macro Feature Subfeature Example (VBA)

This example shows how to create a macro feature subfeature.

```
'-----  
'  
' Preconditions: Model document is open and a sketch is selected.  
'  
' Postconditions: The sketch becomes a subfeature of the newly  
'               created macro feature named EmptyFeature.  
'  
' NOTE: A bitmap named FeatureIcon.bmp for the macro feature must exist.  
'  
'-----  
  
'Macro feature Rebuild routine  
Function swmRebuild(varApp As Variant, varDoc As Variant, varFeat As Variant) As Variant  
    Dim App As SldWorks.SldWorks  
    Dim Doc As SldWorks.ModelDoc2  
    Dim feat As SldWorks.Feature  
    Dim FeatData As SldWorks.MacroFeatureData  
    Dim Rebuild As Boolean  
  
    Set App = varApp  
    Set Doc = varDoc  
    Set feat = varFeat  
  
    MsgBox "Rebuild called"  
  
    Rebuild = True  
    'If an error occurs  
    'Rebuild = "Error message"  
End Function  
  
'Macro feature Edit definition routine  
Function swmEditDefinition(varApp As Variant, varDoc As Variant, varFeat As Variant) As Variant  
    Dim App As SldWorks.SldWorks  
    Dim Doc As SldWorks.ModelDoc2  
    Dim feat As SldWorks.Feature  
    Dim FeatData As SldWorks.MacroFeatureData  
  
    Set App = varApp  
    Set Doc = varDoc  
    Set feat = varFeat  
  
    MsgBox "Edit Definition called"  
  
    Set FeatData = varFeat.GetDefinition  
    varFeat.ModifyDefinition FeatData, varDoc, Nothing  
    EditDefinition = True  
End Function  
  
'Macro feature Security routine  
Function swmSecurity(varApp As Variant, varDoc As Variant, varFeat As Variant) As Variant  
    Dim App As SldWorks.SldWorks  
    Dim Doc As SldWorks.ModelDoc2  
    Dim feat As SldWorks.Feature  
    Dim FeatData As SldWorks.MacroFeatureData  
  
    Set App = varApp  
    Set Doc = varDoc  
    Set feat = varFeat  
  
    swmSecurity = SwConst.swMacroFeatureSecurityOptions_e.swMacroFeatureSecurityByDefault  
End Function  
  
'Base routine to create the macro feature  
Sub CreateSampleFeature()
```

```
Dim swApp As SldWorks.SldWorks
Dim Doc As SldWorks.ModelDoc2
Dim feat As Feature
Dim SelMan As SelectionMgr
```

```
Set swApp = Application.SldWorks
Set Doc = swApp.ActiveDoc
Set SelMan = Doc.SelectionManager
```

```
Dim ThisFile As String
Dim Methods(8) As String
Dim Names As Variant
Dim Types As Variant
Dim Values As Variant
Dim vEditBodies As Variant
Dim options As Long
Dim dimTypes As Variant
Dim dimValue As Variant
Dim icons(2) As String
```

```
'Define the routines to call
ThisFile = swApp.GetCurrentMacroPathName
Methods(0) = ThisFile: Methods(1) = "FeatureModule": Methods(2) = "swmRebuild"
Methods(3) = ThisFile: Methods(4) = "FeatureModule": Methods(5) = "swmEditDefinition"
Methods(6) = "": Methods(7) = "": Methods(8) = "" 'A security routine is optional
```

```
Dim pathname As String
pathname = swApp.GetCurrentMacroPathFolder
icons(0) = pathname + "\FeatureIcon.bmp"
icons(1) = pathname + "\FeatureIcon.bmp"
icons(2) = pathname + "\FeatureIcon.bmp"
```

```
Names = Empty
Types = Empty
Values = Empty
```

```
options = swMacroFeatureByDefault
```

```
Dim selFeat As Feature
Dim swFeatMgr As SldWorks.FeatureManager
Set selFeat = SelMan.GetSelectedObject6(1, -1)
Set swFeatMgr = Doc.FeatureManager
```

```
Set feat = swFeatMgr.InsertMacroFeature3("EmptyFeature", "", (Methods), Names, Types, Values, dimTypes, dimValue, vEditBodies, (icons), options)
```

```
Dim boolstatus As Boolean
boolstatus = feat.MakeSubFeature(selFeat)
End Sub
```

Create a Mate Controller Example (C#)

This example shows how to create a Mate Controller in an assembly.

```
// =====
// Preconditions: Open an assembly document that contains a mechanical slot feature
// with a slot mate that has one of these constraints:
// - Distance Along Slot
// - Percent Along Slot
//
// Postconditions:
// 1. Mate Controller (Position 3) is added to the FeatureManager design tree.
// 2. Inspect the graphics area.
//
// =====
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace SlotMate_CSharp
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 Part;
        private AssemblyDoc Assem;
        private FeatureManager featMgr;
        private bool boolstatus;
        private object myModelView;
        private object[] var;
        private MateControllerFeatureData mateControllerObj;
        private MateControllerFeatureData mcObj2;
        private Feature mateContFeat;
        private double[] position2ValArr = new double[1];
        private double[] position3ValArr = new double[1];
        private object var1;
        private double[] position2ValArr2 = new double[1];
        private double[] position3ValArr2 = new double[1];

        public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
        {
            int ArraySize;
            ArraySize = Objects.GetUpperBound(0);
            Dispatcher[] d = new Dispatcher[ArraySize + 1];
            int ArrayIndex;
            for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
                d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
            return d;
        }

        public void Main()
        {
```

```

Part = (ModelDoc2)swApp.ActiveDoc;
Assem = (AssemblyDoc)Part;

myModelView = Part.ActiveView;

Part.ClearSelection2(true);

boolstatus = Assem.IsSupportedMatesAvailable;

var = (object[])Assem.CollectAllSupportiveMates();

featMgr = Part.FeatureManager;

mateControllerObj = (MateControllerFeatureData)featMgr.CreateDefinition((int)swFeatureNameID_e.swFmMateController);

DispatchWrapper[] dArray;
dArray = ObjectArrayToDispatchWrapperArray(var);

mateControllerObj.Initialize(dArray);

mateControllerObj.AddNewPosition("Position1");
mateControllerObj.AddNewPosition("Position2");
mateControllerObj.AddNewPosition("Position3");

position2ValArr[0] = 0.0375;
mateControllerObj.SetValues("Position2", position2ValArr);

position3ValArr[0] = 0.0625;
mateControllerObj.SetValues("Position3", position3ValArr);

mateContFeat = featMgr.CreateFeature(mateControllerObj);

mcObj2 = (MateControllerFeatureData)mateContFeat.GetDefinition();

var1 = mcObj2.Mates;

position2ValArr2[0] = 0.015;
mcObj2.SetValues("Position2", position2ValArr2);

position3ValArr2[0] = 0.085;
mcObj2.SetValues("Position3", position3ValArr2);

mateContFeat.ModifyDefinition(mcObj2, Part, null);

}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Create a Mate Controller Example (VB.NET)

This example shows how to create a Mate Controller in an assembly.

```
'=====
' Preconditions: Open an assembly document that contains a mechanical slot feature
' with a slot mate that has one of these constraints:
'   - Distance Along Slot
'   - Percent Along Slot
'
' Postconditions:
' 1. Mate Controller (Position 3) is added to the FeatureManager design tree.
' 2. Inspect the graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
```

```
    Dim Assem As AssemblyDoc
```

```
    Dim featMgr As FeatureManager
```

```
    Dim boolstatus As Boolean
```

```
    Dim myModelView As Object
```

```
    Dim var As Object
```

```
    Dim mateControllerObj As MateControllerFeatureData
```

```
    Dim mcObj2 As MateControllerFeatureData
```

```
    Dim mateContFeat As Feature
```

```
    Dim position2ValArr(0) As Double
```

```
    Dim position3ValArr(0) As Double
```

```
    Dim var1 As Object
```

```
    Dim position2ValArr2(0) As Double
```

```
    Dim position3ValArr2(0) As Double
```

```
Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
```

```
    Dim ArraySize As Integer
```

```
    ArraySize = Objects.GetUpperBound(0)
```

```
    Dim d(ArraySize) As DispatchWrapper
```

```
    Dim ArrayIndex As Integer
```

```
    For ArrayIndex = 0 To ArraySize
```

```
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
```

```
    Next
```

```
    Return d
```

```
End Function
```

```
Sub main()
```

```
    Part = swApp.ActiveDoc
```

```
    Assem = Part
```

```
    myModelView = Part.ActiveView
```

```
    myModelView.FrameState = swWindowState_e.swWindowMaximized
```

```
    Part.ClearSelection2(True)
```

```
    boolstatus = Assem.IsSupportedMatesAvailable
```

```

var = Assem.CollectAllSupportiveMates

featMgr = Part.FeatureManager

mateControllerObj = featMgr.CreateDefinition(swFeatureNameID_e.swFmMateController)

Dim dArray() As DispatchWrapper
dArray = ObjectArrayToDispatchWrapperArray(var)

Call mateControllerObj.Initialize(dArray)

mateControllerObj.AddNewPosition("Position1")
mateControllerObj.AddNewPosition("Position2")
mateControllerObj.AddNewPosition("Position3")

position2ValArr(0) = 0.0375
Call mateControllerObj.SetValues("Position2", position2ValArr)

position3ValArr(0) = 0.0625
Call mateControllerObj.SetValues("Position3", position3ValArr)

mateContFeat = featMgr.CreateFeature(mateControllerObj)

mcObj2 = mateContFeat.GetDefinition()

var1 = mcObj2.Mates

position2ValArr2(0) = 0.015
Call mcObj2.SetValues("Position2", position2ValArr2)

position3ValArr2(0) = 0.085
Call mcObj2.SetValues("Position3", position3ValArr2)

mateContFeat.ModifyDefinition(mcObj2, Part, Nothing)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create a Callout in a Model View Example (C#)

This example shows how to create a callout in a model view.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Select:
//    Project > Add Reference > .NET > SolidWorks.Interop.swpublished
//
// Postconditions:
// 1. A callout with one row is created in the first model view.
// 2. Press F5 three more times to add a callout to three other model views.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using System;
using System.Runtime.InteropServices;
using System.Diagnostics;
namespace ModelViewCallout_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModelDoc;
        ModelDocExtension swModelDocExtn;
        ModelViewManager ViewMgr;
        ModelView swModelView;
        Callout Viewcallout;

        CalloutHandler handle = new CalloutHandler();

        public void Main()
        {
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExtn = swModelDoc.Extension;

            ViewMgr = swModelDoc.ModelViewManager;
            ViewMgr.ViewportDisplay = (int)swViewportDisplay_e.swViewportFourView;
            swModelDoc.GetModelViewCount();
            swModelView = (ModelView)swModelDoc.GetFirstModelView();
            while ((swModelView != null))
            {
                Viewcallout = swModelView.CreateCallout(1, handle);
                Viewcallout.set_Label2(0, "TEST");
                Viewcallout.SkipColon = false;
                Viewcallout.set_ValueInactive(0, true);
                Viewcallout.SetTargetPoint(0, 0.0, 0.0, 0.0);
                Viewcallout.Display(true);
                System.Diagnostics.Debugger.Break();
                swModelView = (ModelView)swModelView.GetNext();
            }
        }

        public SldWorks swApp;
    }

    [ComVisibleAttribute(true)]
    public class CalloutHandler : SwCalloutHandler
    {
        public bool OnStringValueChanged(object pManipulator, int RowID, string Text)
        {
            Debug.Print("Text: " + Text);
            Debug.Print("Row: " + RowID);

            return true;
        }
    }
}
```

Create a Callout in a Model View Example (VBA)

This example shows how to create a callout in a model view.

```
'-----
' Preconditions:
' 1. Open a part document.
' 2. Select:
'   Tools > References > SOLIDWORKS <version> exposed type
'   Libraries for add-in use.
' 3. Copy Class1 to a class module named CalloutHandler.
'
' Postconditions:
' 1. A callout with one row is created in the first model view.
' 2. Press F5 three more times to add a callout to three other model views.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swModelDocExtn As SldWorks.ModelDocExtension
Dim ViewMgr As ModelViewManager
Dim swModelView As ModelView
Dim Viewcallout As SldWorks.Callout
Dim handle As New CalloutHandler

Sub main()
    Set swApp = Application.SldWorks
    Set swModelDoc = swApp.ActiveDoc
    Set swModelDocExtn = swModelDoc.Extension

    Set ViewMgr = swModelDoc.ModelViewManager
    ViewMgr.ViewportDisplay = swViewportDisplay_e.swViewportFourView
    swModelDoc.GetModelViewCount
    Set swModelView = swModelDoc.GetFirstModelView
    While (Not swModelView Is Nothing)
        Set Viewcallout = swModelView.CreateCallout(1, handle)
        Viewcallout.Label2(0) = "TEST"
        Viewcallout.SkipColon = False
        Viewcallout.ValueInactive(0) = True
        Call Viewcallout.SetTargetPoint(0, 0#, 0#, 0#)
        Viewcallout.Display (True)
        Stop
        Set swModelView = swModelView.GetNext
    Wend
End Sub
```

[Class1](#):

Implements SwCalloutHandler

```
Private Function SwCalloutHandler_OnStringValueChanged(ByVal pManipulator As Object, ByVal RowID As Long, ByVal Text As String) As Boolean
```

```
    Debug.Print ("Text: " & Text)
    Debug.Print ("Row: " & RowID)
    SwCalloutHandler_OnStringValueChanged = True
```

```
End Function
```

Create a Callout in a Model View Example (VB.NET)

This example shows how to create a callout in a model view.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Select:
'   Project > Add Reference > .NET > SolidWorks.Interop.swpublished
' 3. Copy #Class1: to a class named CalloutHandler.
'
' Postconditions:
' 1. A callout with one row is created in the first model view.
' 2. Press F5 three more times to add a callout to three other model views.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

```

```

Partial Class SolidWorksMacro

```

```

    Dim swModelDoc As ModelDoc2
    Dim swModelDocExtn As ModelDocExtension
    Dim ViewMgr As ModelViewManager
    Dim swModelView As ModelView
    Dim Viewcallout As Callout
    Dim handle As New CalloutHandler

```

```

    Sub main()

```

```

        swModelDoc = swApp.ActiveDoc
        swModelDocExtn = swModelDoc.Extension

```

```

        ViewMgr = swModelDoc.ModelViewManager
        ViewMgr.ViewportDisplay = swViewportDisplay_e.swViewportFourView
        swModelDoc.GetModelViewCount()
        swModelView = swModelDoc.GetFirstModelView
        While (Not swModelView Is Nothing)
            Viewcallout = swModelView.CreateCallout(1, handle)
            Viewcallout.Label2(0) = "TEST"
            Viewcallout.SkipColon = False
            Viewcallout.ValueInactive(0) = True
            Call Viewcallout.SetTargetPoint(0, 0.0#, 0.0#, 0.0#)
            Viewcallout.Display(True)
            Stop
            swModelView = swModelView.GetNext
        End While

```

```

    End Sub

```

```

    Public swApp As SldWorks

```

```

End Class

```

```

Class1:

```

```

Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Runtime.InteropServices
Imports System.Diagnostics
<ComVisibleAttribute(True)> _
Public Class CalloutHandler
    Implements SwCalloutHandler

```

```

    Public Function OnStringValueChanged(ByVal pManipulator As Object, ByVal RowID As Integer, ByVal Text As String) As Boolean
    Implements SolidWorks.Interop.swpublished.SwCalloutHandler.OnStringValueChanged

```

```

        Debug.Print("Text: " & Text)
        Debug.Print("Row: " & RowID)

```

```

        Return True

```

```

    End Function

```

```

End Class

```

Create Motion Studies Example (VBA)

This example shows how to create, rename, and delete motion studies.

```

'-----
' Preconditions:
' 1. Open a part or assembly that does not have any user-created
'    motion studies.
' 2. If the MotionManager tab is not visible, click View > MotionManager.
' 3. Add a reference to the SOLIDWORKS MotionStudy type library.
'
' Postconditions:
' 1. Creates two motion studies, deletes one of them, and
'    renames one of them.
' 2. Examine the lower-portion of the graphics area and the
'    Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMotionMgr As SwMotionStudy.MotionStudyManager
Dim swMotionStudy1 As SwMotionStudy.MotionStudy
Dim swMotionStudy2 As SwMotionStudy.MotionStudy
Dim swMotionStudy3 As SwMotionStudy.MotionStudy
Dim swSaveAVIData As SwMotionStudy.AVIParameter
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    If (swMotionMgr Is Nothing) Then
        End
    End If

    ' Create new motion studies
    Set swMotionStudy2 = swMotionMgr.CreateMotionStudy()
    If (swMotionStudy2 Is Nothing) Then
        MsgBox "Motion Study 2 is not available."
        End
    End If

    Set swMotionStudy3 = swMotionMgr.CreateMotionStudy()
    If (swMotionStudy3 Is Nothing) Then
        MsgBox "Motion Study 3 is not available."
        End
    End If

    ' Get number of motion studies
    Debug.Print "Number of motion studies = " & swMotionMgr.GetMotionStudyCount

    ' Get motion study
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")
    If (swMotionStudy1 Is Nothing) Then
        MsgBox "Motion Study 1 is not available."
        End
    End If

    ' Get number and names of motion studies and whether they've been activated
    Dim vNames As Variant
    Dim i As Long
    vNames = swMotionMgr.GetMotionStudyNames()
    For i = 0 To UBound(vNames)
        Debug.Print vNames(i);
        boolstatus = swMotionMgr.ActivateMotionStudy(vNames(i))
        Debug.Print " activated = " & boolstatus
    Next i

    ' Attempt to activate a non-existing motion study
    boolstatus = swMotionMgr.ActivateMotionStudy("Motion Study 4") ' Should return False
    Debug.Print "Motion Study 4 activated = " & boolstatus

    ' Delete Motion Study2 and non-existing motion study
    boolstatus = swMotionMgr.DeleteMotionStudy("Motion Study 2")
    Debug.Print "Motion Study 2 deleted = " & boolstatus
    boolstatus = swMotionMgr.DeleteMotionStudy("Motion Study 4")
    Debug.Print "Motion Study 4 deleted = " & boolstatus ' Should return False

    ' Rename Motion Study 3 to TestName
    swMotionStudy3.Name = "TestName"

    ' Set and save AVI parameters
    Set swSaveAVIData = swMotionMgr.CreateAVIParameter()
    swSaveAVIData.FramePerSecond = 7.5
    swSaveAVIData.SaveEntireAnimation = True
    swSaveAVIData.OutputType = 1 ' Save as an .avi file

End Sub

```

Create Multi-row Callouts Example (VBA)

This example shows how to create multi-row callouts.

```
'-----
' Preconditions:
' 1. Copy and paste this code in the main module.
' 2. Click Insert > Class module and copy and paste
'    this code in the class module.
' 3. Click Tools > References > SOLIDWORKS version
'    exposed type libraries for add-in use.
' 4. Open a part or a fully resolved assembly.
' 5. Select one or more geometric entities in
'    the graphics area.
'
' Postconditions: Observe the graphics area while stepping
'                 through the macro (put your cursor in the main module
'                 in the IDE and press F8 repeatedly) to verify that callouts
'                 are attached to each geometric entity selected in
'                 Preconditions step 5.
'
' NOTE: Selecting features in FeatureManager design tree is
'       not currently supported.
'-----
'Main module

Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim vSelPos() As Variant
    Dim swCallout() As SldWorks.Callout
    Dim nSelCount As Long
    Dim i As Long
    Dim bRet As Boolean
    Dim callH As New Class1
    Dim boolstatus As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swModelExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    nSelCount = swSelMgr.GetSelectedObjectCount2(-1)

    ReDim vSelPos(nSelCount)
    ReDim swCallout(nSelCount)

    For i = 0 To nSelCount - 1
        vSelPos(i) = swSelMgr.GetSelectionPoint2((i + 1), -1)
    Next i

    swModel.ClearSelection2 True

    For i = 0 To nSelCount - 1
        Set swCallout(i) = swSelMgr.CreateCallout2(4, callH)
        swCallout(i).Label1(0) = "Project"
        swCallout(i).Label1(1) = "Product"
        swCallout(i).Label1(2) = "Radius"
        swCallout(i).Label1(3) = "Number"
        swCallout(i).Value(0) = "10685"
        swCallout(i).Value(1) = "Washer"
        swCallout(i).Value(2) = "30"
        swCallout(i).Value(3) = "1"
        swCallout(i).SetTargetPoint 0, vSelPos(i)(0), vSelPos(i)(1), vSelPos(i)(2)
        swCallout(i).SetTargetPoint 1, vSelPos(i)(0), vSelPos(i)(1), vSelPos(i)(2)
        swCallout(i).SetTargetPoint 2, vSelPos(i)(0), vSelPos(i)(1), vSelPos(i)(2)
        swCallout(i).SetTargetPoint 3, vSelPos(i)(0), vSelPos(i)(1), vSelPos(i)(2)
        swCallout(i).TextColor(0) = swSystemColorsRefTriadX
        swCallout(i).TextColor(1) = swSystemColorsRefTriadY
        swCallout(i).TextColor(2) = swSystemColorsRefTriadZ
        swCallout(i).TextColor(3) = swSystemColorsRefTriadX
        swCallout(i).OpaqueColor = swSystemColorsSelectedItem4
        swCallout(i).MultipleLeaders = False
        swCallout(i).EnablePushPin = True
        bRet = swModelExt.SelectByID2("", "", vSelPos(i)(0), vSelPos(i)(1), vSelPos(i)(2), True, 0, swCallout(i), 0)
        swCallout(i).ValueInactive(3) = True
    Next i

End Sub
```

[Back to top](#)

```
'Class module

Option Explicit

Implements SwCalloutHandler

Dim m_pCallout As SldWorks.Callout

Public Sub Init(clout As SldWorks.Callout)
    Set m_pCallout = clout
```

```
End Sub

Private Sub Class_Initialize()
    Debug.Print "Class_Initialize"
End Sub

Private Sub Class_Terminate()
    Debug.Print "Class_Terminate"
End Sub

Public Function SwCalloutHandler_OnStringValueChanged(ByVal pManipulator As Object, ByVal Index As Long, ByVal Text As String) As Boolean
    Debug.Print Index
    Debug.Print Text
    Dim retval As Boolean
End Function
```

[Back to top](#)

Create Multibody Macro Feature Example (C#)

This example shows how to create a multibody macro feature using a Visual C# SOLIDWORKS add-in.

```
//-----
// Preconditions:
// 1. Contact SOLIDWORKS API Support to obtain C#\VB.NET Add-ins to Create a Macro Feature.
// 2. Copy SwCSharpAddinMF.zip to a local drive.
// 3. Unzip the file.
// 4. Open Visual Studio 2015 and convert the Visual C# solution in Visual Studio.
// 5. Modify the project references to point to your SOLIDWORKS primary
//    interop assemblies.
// 6. Right-click the project name in the Solution Explorer, click
//    Add Reference, click Assemblies > Framework, click
//    Microsoft.VisualBasic, select its check box, click Microsoft.CSharp, select
//    its check box, and click OK.
// 7. Double-click SwAddin.cs in the Solution Explorer.
// 8. Replace region ISwComFeature Implementation with this.
// 9. Replace AddMacroFeature() with this.
// 10. Click Project > SwCSharpAddinMF Properties > Application and change
//    Target framework to .NET Framework 4.
// 11. Add using Microsoft.VisualBasic; to the top of SwAddIn.cs.
// 12. Click Build > Build Solution.
//    NOTE: The Output window notifies that types registered successfully.
//    If that message does not appear in the Output window, repeat from
//    step 1.
// 13. Open SOLIDWORKS Design.
// 14. Open public_documents\samples\tutorial\multibody\multi_local.sldprt.
//
// Postconditions:
// 1. Click OK in each message box.
// 2. Displays C# Addin on the Tools menu.
// 3. Fires the ISldWorks_ActiveDocChangeNotify event and inserts
//    the macro feature, MacroFeature1, in the FeatureManager design tree,
//    which:
//    * Consumes the part's solid bodies, Fillet5 and Fillet6.
//    * Creates two solid bodies, MacroFeature1[1] and MacroFeature1[2].
// 4. Examine the graphics area and FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

//Region ISwComFeature Implementation
#region "ISwComFeature Implementation"

    public object Edit(object app, object modelDoc, object feature)
    {
        Interaction.MsgBox("Macro feature edit");
        return null;
    }

    public object Regenerate(object app, object modelDoc, object feature)
    {
        object functionReturnValue = null;
        Collection OutputBodies = new Collection();
        Body2 swBody = default(Body2);
        Body2[] swBodies = null;
        MacroFeatureData swMacroFeatData = default(MacroFeatureData);
        swMacroFeatData = (MacroFeatureData)((Feature)feature).GetDefinition();
        swMacroFeatData.EnableMultiBodyConsume = true;

        Modeler swModeler = default(Modeler);
        swModeler = (Modeler)((SldWorks)app).GetModeler();
        double[] dblData = new double[9];
        dblData[0] = 0;
        dblData[1] = 0;
        dblData[2] = 0;
        dblData[3] = 1;
        dblData[4] = 0;
        dblData[5] = 0;
        dblData[6] = 0.1;
        dblData[7] = 0.1;
        dblData[8] = 0.1;

        //Output body 1
        swBody = swModeler.CreateBodyFromBox3(dblData);
        OutputBodies.Add(swBody);

        //Output body 2
        dblData[1] = 0.15;
        swBody = swModeler.CreateBodyFromBox3(dblData);
        OutputBodies.Add(swBody);

        int i = 0;
        int j = 0;
        object[] vFaces = null;
        object[] vEdges = null;

        for (i = 1; i <= OutputBodies.Count; i++)
```

```

    {
        swBody = (Body2)OutputBodies[i];
        vEdges = (object[])swBody.GetEdges();
        vFaces = (object[])swBody.GetFaces();

        for (j = 0; j <= Information.UBound(vEdges); j++)
        {
            swMacroFeatData.SetEdgeUserId((Edge)vEdges[j], j, 0);
        }
        for (j = 0; j <= Information.UBound(vFaces); j++)
        {
            swMacroFeatData.SetFaceUserId((Face2)vFaces[j], j, 0);
        }

        swBodies[i - 1] = (Body2)OutputBodies[i];
    }

    functionReturnValue = swBodies;
    Interaction.MsgBox("Macro feature rebuild");
    return functionReturnValue;
}

public object Security(object app, object modelDoc, object feature)
{
    Interaction.MsgBox("Macro feature security");
    return null;
}

#endregion

```

[Back to top](#)

//AddMacroFeature

```

public bool AddMacroFeature()
{
    ModelDoc2 moddoc = default(ModelDoc2);
    FeatureManager FeatMgr = default(FeatureManager);
    Feature MacroFeature = default(Feature);

    moddoc = (ModelDoc2)this.iSwApp.ActiveDoc;
    FeatMgr = moddoc.FeatureManager;

    //Collect input bodies
    object vBodies = null;
    vBodies = ((PartDoc)moddoc).GetBodies2((int)swBodyType_e.swAllBodies, false);

    //Create the macro feature
    MacroFeature = FeatMgr.InsertMacroFeature3("MacroFeature", "SwVBAddinTest.SwAddin", null, null, null, null, null, null, vBodies, null, (int)swMacroFeatureOptions_e.swMacroFeatureByDefault);

    return true;
}

```

[Back to top](#)

Create Multibody Macro Feature Example (VBA)

This example shows how to create a multibody macro feature.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_local.sldprt.
' 2. Copy main module to the macro code window.
' 3. Right-click the project name and click Insert > Module.
' 4. Type mMacroFeature in (Name) in the module's Properties window
'    (if necessary, right-click Module1 to display the Properties window).
' 5. Copy macro feature to the mMacroFeature code window.
'
' Postconditions:
' 1. Creates MacroFeature1, which:
'    * Consumes the part's solid bodies, Fillet5 and Fillet6.
'    * Creates two solid bodies, MacroFeature1[1] and MacroFeature1[2].
' 2. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

'main module

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swPart As SldWorks.PartDoc
Option Explicit

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel

    Dim strMacroMethods(8) As String
    'Rebuild function
    strMacroMethods(0) = swApp.GetCurrentMacroPathName
    strMacroMethods(1) = "mMacroFeature"
    strMacroMethods(2) = "swmRebuild"
    'Edit definition function
    strMacroMethods(3) = swApp.GetCurrentMacroPathName
    strMacroMethods(4) = "mMacroFeature"
    strMacroMethods(5) = "swmEditDefinition"
    'Security function
    strMacroMethods(6) = swApp.GetCurrentMacroPathName
    strMacroMethods(7) = "mMacroFeature"
    strMacroMethods(8) = "swmSecurity"

    'Collect input bodies
    Dim vBodies As Variant
    vBodies = swPart.GetBodies2(swAllBodies, False)

    'Create the macro feature
    swModel.FeatureManager.InsertMacroFeature3 "MacroFeature", "", strMacroMethods, _
        Nothing, Nothing, Nothing, Nothing, Nothing, vBodies, Nothing, swMacroFeatureByDefault
End Sub

```

[Go to top](#)

```

'macro feature

Function swmRebuild(app As Variant, model As Variant, feat As Variant) As Variant
    Dim OutputBodies As New Collection
    Dim swBody As SldWorks.Body2
    Dim swBodies() As SldWorks.Body2
    Dim swMacroFeatData As SldWorks.MacroFeatureData
    Set swMacroFeatData = feat.GetDefinition
    swMacroFeatData.EnableMultiBodyConsume = True

    Dim swModeler As SldWorks.Modeler
    Set swModeler = app.GetModeler
    Dim dblData(8) As Double
    dblData(0) = 0: dblData(1) = 0: dblData(2) = 0
    dblData(3) = 1: dblData(4) = 0: dblData(5) = 0
    dblData(6) = 0.1: dblData(7) = 0.1: dblData(8) = 0.1

    'Output body 1
    Set swBody = swModeler.CreateBodyFromBox3(dblData)
    OutputBodies.Add swBody

    'Output body 2
    dblData(1) = 0.15
    Set swBody = swModeler.CreateBodyFromBox3(dblData)
    OutputBodies.Add swBody

    Dim i As Integer, j As Integer
    Dim vFaces As Variant

```

```

Dim vEdges As Variant
ReDim swBodies(OutputBodies.Count - 1)

For i = 1 To OutputBodies.Count
    Set swBody = OutputBodies.Item(i)
    vEdges = swBody.GetEdges
    vFaces = swBody.GetFaces
    For j = 0 To UBound(vEdges)
        swMacroFeatData.SetEdgeUserId vEdges(j), j, 0
    Next j
    For j = 0 To UBound(vFaces)
        swMacroFeatData.SetFaceUserId vFaces(j), j, 0
    Next j
    Set swBodies(i - 1) = OutputBodies.Item(i)
Next i
swmRebuild = swBodies

End Function

Function swmEditDefinition(app As Variant, model As Variant, feat As Variant) As Variant
End Function

Function swmSecurity(app As Variant, model As Variant, feat As Variant) As Variant
End Function

```

[Go to top](#)

Create Multibody Macro Feature Example (VB.NET)

This example shows how to create a multibody macro feature using a VB.NET SOLIDWORKS add-in.

```

-----
' Preconditions:
' 1. Contact SOLIDWORKS API Support to obtain API: C#\VB.NET Add-ins to Create a Macro Feature.
' 2. Save InsertMacroFeature VB.NET Addin Mods for 64 bit.zip to a local drive.
' 3. Unzip the file.
' 4. Open Visual Studio 2015 and convert the VB.NET solution in Visual Studio.
' 5. Modify the project references to point to your SOLIDWORKS primary interop assemblies.
' 6. Double-click SwAddin.vb in the Solution Explorer.
' 7. Replace Region ISwComFeature Implementation with this.
' 8. Replace AddMacroFeature() with this.
' 9. Click Project > SwVBAddinTest Properties > Application and change Target framework to .NET Framework 4.
' 10. Click Compile > Build Events and replace the code in Post-build event command line with this.
' 11. Click Build > Build Solution.
'     NOTE: The Output window notifies that types registered successfully.
'     If that message does not appear in the Output window, repeat from step 1.
' 12. Open SOLIDWORKS Design.
' 13. Open public_documents\samples\tutorial\multibody\multi_local.sldprt.
'
' Postconditions:
' 1. Click OK in each message box.
' 2. Displays VB Addin on the Tools menu.
' 3. Fires the ISldWorks_ActiveDocChangeNotify event and inserts the macro feature, MacroFeature1, in the FeatureManager design tree, which:
'     * Consumes the part's solid bodies, Fillet5 and Fillet6.
'     * Creates two solid bodies, MacroFeature1[1] and MacroFeature1[2].
' 4. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save changes.
-----

'Region ISwComFeature Implementation
#Region "ISwComFeature Implementation"

Function Edit(ByVal app As Object, ByVal modelDoc As Object, ByVal feature As Object) As Object Implements SwComFeature.Edit
    MsgBox("Macro feature edit")
    Return Nothing
End Function

Function Regenerate(ByVal app As Object, ByVal modelDoc As Object, ByVal feature As Object) As Object Implements SwComFeature.Re
    Dim OutputBodies As New Collection
    Dim swBody As Body2
    Dim swBodies() As Body2
    Dim swMacroFeatData As MacroFeatureData
    swMacroFeatData = feature.GetDefinition
    swMacroFeatData.EnableMultiBodyConsume = True

    Dim swModeler As Modeler
    swModeler = app.GetModeler
    Dim dblData(8) As Double
    dblData(0) = 0 : dblData(1) = 0 : dblData(2) = 0
    dblData(3) = 1 : dblData(4) = 0 : dblData(5) = 0
    dblData(6) = 0.1 : dblData(7) = 0.1 : dblData(8) = 0.1

    'Output body 1
    swBody = swModeler.CreateBodyFromBox3(dblData)
    OutputBodies.Add(swBody)

    'Output body 2
    dblData(1) = 0.15
    swBody = swModeler.CreateBodyFromBox3(dblData)
    OutputBodies.Add(swBody)

    Dim i As Integer, j As Integer
    Dim vFaces As Object
    Dim vEdges As Object
    ReDim swBodies(OutputBodies.Count - 1)

    For i = 1 To OutputBodies.Count
        swBody = OutputBodies.Item(i)
        vEdges = swBody.GetEdges
        vFaces = swBody.GetFaces

        For j = 0 To UBound(vEdges)
            swMacroFeatData.SetEdgeUserId(vEdges(j), j, 0)
        Next j
        For j = 0 To UBound(vFaces)
            swMacroFeatData.SetFaceUserId(vFaces(j), j, 0)
        Next j

        swBodies(i - 1) = OutputBodies.Item(i)
    Next i

```

```

        Next i

        Regenerate = swBodies
        MsgBox("Macro feature rebuild")

    End Function

    Function Security(ByVal app As Object, ByVal modelDoc As Object, ByVal feature As Object) As Object Implements SwComFeature.Security
        MsgBox("Macro feature security")
        Return Nothing
    End Function

#End Region

```

[Back to top](#)

'AddMacroFeature

```
Function AddMacroFeature() As Boolean
```

```

    Dim moddoc As ModelDoc2
    Dim FeatMgr As FeatureManager
    Dim MacroFeature As Feature

```

```

    moddoc = Me.iSwApp.ActiveDoc
    FeatMgr = moddoc.FeatureManager

```

```

'Collect input bodies
Dim vBodies As Object
vBodies = moddoc.GetBodies2(swBodyType_e.swAllBodies, False)

```

```

'Create the macro feature
MacroFeature = FeatMgr.InsertMacroFeature3("MacroFeature", "swVBAddinTest.SwAddin", Nothing, _
    Nothing, Nothing, Nothing, Nothing, Nothing, vBodies, Nothing, swMacroFeatureOptions_e.swMacroFeatureByDefault)

```

```
End Function
```

[Back to top](#)

```

IF EXIST "%(TargetDir)%(TargetName).bmp" (GOTO REGISTRATION)
XCOPY "%(ProjectDir)AddinIcon.bmp" "%(TargetDir)" /F
REN "%(TargetDir)AddinIcon.bmp" "%(TargetName).bmp"

```

```

:REGISTRATION
IF "%(TargetFrameworkVersion)"=="v4.0" GOTO NET40
IF "%(TargetFrameworkVersion)"=="v3.5" GOTO NET20
IF "%(TargetFrameworkVersion)"=="v3.0" GOTO NET20
IF "%(TargetFrameworkVersion)"=="v2.0" GOTO NET20
GOTO END

```

```

:NET40
set FMWK="v4.0.30319"
GOTO REG

```

```

:NET20
set FMWK="v2.0.50727"
GOTO REG

```

```

:REG
IF "%(PlatformName)" == "AnyCPU" GOTO ANYCPU
IF "%(PlatformName)" == "x64" GOTO X64
GOTO END

```

```

:ANYCPU
IF EXIST "%windir%\Microsoft.NET\Framework64\FMWK%\regasm.exe" "%windir%\Microsoft.NET\Framework64\FMWK%\regasm" /codebase
"%(TargetPath)"
GOTO END

```

```

:X64
IF EXIST "%windir%\Microsoft.NET\Framework64\FMWK%\regasm.exe" "%windir%\Microsoft.NET\Framework64\FMWK%\regasm" /codebase
"%(TargetPath)"
GOTO END

```

```
:END
```

[Back to top](#)

Create Multiple-Radius Fillets Example (C#)

This example shows how to create multiple-radius fillets.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\box.sldprt.
// 2. Select three intersecting edges on the inside of the part.
//
// Postconditions:
// 1. Fillets the three selected edges using three
//    different radii.
// 2. Examine the graphics area and FeatureManager design tree.
//
// Note: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace FilletFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        FeatureManager swFeatMgr;
        object VarRadArray8;
        double[] radArray8 = new double[3];

        int FilletOptions;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swFeatMgr = swModel.FeatureManager;

            //Multiple Radii
            radArray8[0] = 0.01;
            radArray8[1] = 0.015;
            radArray8[2] = 0.02;
            VarRadArray8 = radArray8;

            //Fillet options
            FilletOptions = (int)swFeatureFilletOptions_e.swFeatureFilletPropagate +
            (int)swFeatureFilletOptions_e.swFeatureFilletAttachEdges + (int)swFeatureFilletOptions_e.swFeatureFilletKeepFeatures;

            //Create multiple-radius fillets along selected edges
            swFeatMgr.FeatureFillet(FilletOptions, 0.01, (int)swFeatureFilletType_e.swFeatureFilletType_Simple,
            (int)swFilletOverflowType_e.swFilletOverflowType_Default, (VarRadArray8), 0, 0);

        }

        public SldWorks swApp;
    }
}
```

Create Multiple-Radius Fillets Example (VBA)

This example shows how to create multiple-radius fillets.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\box.sldprt.  
' 2. Select three intersecting edges on the inside of the part.  
'  
' Postconditions:  
' 1. Fillets the three selected edges using three  
'    different radii.  
' 2. Examine the graphics area and FeatureManager design tree.  
'  
' Note: Because the model is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swFeatMgr As SldWorks.FeatureManager  
Dim VarRadArray8 As Variant  
Dim radArray8(2) As Double  
Dim FilletOptions As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swFeatMgr = swModel.FeatureManager  
  
    'Multiple Radii  
    radArray8(0) = 0.01  
    radArray8(1) = 0.015  
    radArray8(2) = 0.02  
    VarRadArray8 = radArray8  
  
    'Fillet options  
    FilletOptions = swFeatureFilletPropagate + swFeatureFilletAttachEdges + swFeatureFilletKeepFeatures  
  
    'Create multiple-radius fillets along selected edges  
    swFeatMgr.FeatureFillet FilletOptions, 0.01, swFeatureFilletType_Simple, swFilletOverflowType_Default, (VarRadArray8), 0, 0  
  
End Sub
```

Create Multiple-Radius Fillets Example (VB.NET)

This example shows how to create multiple-radius fillets.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\box.sldprt.
' 2. Select three intersecting edges on the inside of the part.
'
' Postconditions:
' 1. Fillets the three selected edges using three
'    different radii.
' 2. Examine the graphics area and FeatureManager design tree.
'
' Note: Because the model is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swFeatMgr As FeatureManager
    Dim VarRadArray8 As Object
    Dim radArray8(2) As Double
    Dim FilletOptions As Long
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swFeatMgr = swModel.FeatureManager
```

```
        'Multiple Radii
        radArray8(0) = 0.01
        radArray8(1) = 0.015
        radArray8(2) = 0.02
        VarRadArray8 = radArray8
```

```
        'Fillet options
```

```
        FilletOptions = swFeatureFilletOptions_e.swFeatureFilletPropagate + swFeatureFilletOptions_e.swFeatureFilletAttachEdges +
        swFeatureFilletOptions_e.swFeatureFilletKeepFeatures
```

```
        'Create multiple-radius fillets along selected edges
```

```
        swFeatMgr.FeatureFillet(FilletOptions, 0.01, swFeatureFilletType_e.swFeatureFilletType_Simple,
        swFilletOverflowType_e.swFilletOverflowType_Default, (VarRadArray8), 0, 0)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Create Hidden Undo Object Example (C#)

This example shows how to create an Undo object that is hidden in the SOLIDWORKS Undo list.

```
//-----
// Preconditions: Ensure the part template path exists.
//
// Postconditions:
// 1. A part with four sketches is created.
// 2. One sketch is extruded.
// 3. A hidden Undo object, API Undo, is created with two extrusions.
// 4. One sketch is cut extruded.
// 5. The following items appear in the SOLIDWORKS Undo list in this order:
//   a. Extruded Cut
//   b. (API Undo, hidden from view)
//   c. Base
//
// NOTE: If you select Base in the SOLIDWORKS Undo list:
// 1. The base boss created before the recording of the hidden API Undo object is undone.
// 2. The two bosses created during the recording of the hidden API Undo object are undone.
// 3. The cut extrusion created after the recording of the hidden API Undo object is undone.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace HiddenObjectUndoCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelView swModelView;
        ModelDocExtension swModelDocExt;
        SketchManager swSketchManager;
        SketchSegment swSketchSegment;
        Feature swFeature;
        FeatureManager swFeatureManager;
        SelectionMgr swSelectionManager;
        bool status;
        int docErrors;

        public void Main()
        {
            swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2013\\templates\\Part.prt", (int)swDwgPaperSizes_e.swDwgPaperASize, 0, 0);
            swApp.ActivateDoc3("Part2.sldprt", false, (int)swRebuildOnActivation_e.swDontRebuildActiveDoc, ref docErrors);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swModelDocExt = (ModelDocExtension)swModel.Extension;

            swModelView = (ModelView)swModel.ActiveView;
            swModelView.FrameState = (int)swWindowState_e.swWindowMaximized;

            swSketchManager = (SketchManager)swModel.SketchManager;

            swSketchManager.InsertSketch(true);
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0692248508634211, 0.0392379182115397, 0.00987134779060705, false, 0, null, 0);
            swModel.ClearSelection2(true);

            object vSkLines = null;
            vSkLines = swSketchManager.CreateCornerRectangle(-0.0891172006155176, 0.0314069429482, 0, -0.0425302352423542, 0.00601966406507166, 0);
            swModel.ClearSelection2(true);

            swSketchManager.InsertSketch(true);
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);

            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.009029, 0.03036, 0.0, 0.021854, 0.019629, 0.0);
            swModel.ClearSelection2(true);

            swSketchManager.InsertSketch(true);
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);

            swSketchSegment = (SketchSegment)swSketchManager.CreateEllipse(0.0306284568434307, 0.00619756829649987, 0, 0.0309763470298606, 0.00997419305453208, 0, 0.0286971648691861, 0.00637547252792807, 0);
            swModel.ClearSelection2(true);

            swSketchManager.InsertSketch(true);
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);

            swSketchSegment = (SketchSegment)swSketchManager.CreateEllipse(0.0240620641310443, 0.0131240684851264, 0, 0.0771974468433887, 0.0706711158113391, 0, 0.000886560440335415, 0.0345228945826079, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);

            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true, 0, 0, false);
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            swSelectionManager.EnableContourSelection = false;

            // Start recording the SOLIDWORKS Undo object
            swModelDocExt.StartRecordingUndoObject();
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 4, null, 0);
            swFeature = (Feature)swFeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true, 0, 0, false);
            swSelectionManager.EnableContourSelection = false;
            status = swModelDocExt.SelectByID2("Sketch4", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swFeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true, 0, 0, false);
            swSelectionManager.EnableContourSelection = false;
        }
    }
}
```

```
// End recording the SOLIDWORKS Undo object with name "API Undo" and hide it in the Undo list
swModelDocExt.FinishRecordingUndoObject2("API Undo", true);
swModelDocExt.ClearSelection2(true);
status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, false, 4, null, 0);
swFeature = (Feature)swFeatureManager.FeatureCut3(true, false, true, 0, 0, 0.00254, 0.00254, false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, true, true,
true, true, false, 0, 0, false);
swSelectionManager.EnableContourSelection = false;

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
```

Create Hidden Undo Object Example (VBA)

This example shows how to create an Undo object that is hidden in the SOLIDWORKS Undo list.

```

'-----
' Preconditions: Ensure the part template path exists.
'
' Postconditions:
' 1. A part with four sketches is created.
' 2. One sketch is extruded.
' 3. A hidden Undo object, API Undo, is created with two extrusions.
' 4. One sketch is cut extruded.
' 5. The following items appear in the SOLIDWORKS Undo list in this order:
'   a. Extruded Cut
'   b. (API Undo, hidden from view)
'   c. Base
'
' NOTE: If you select Base in the SOLIDWORKS Undo list:
' 1. The base boss created before the recording of the hidden API Undo object is undone.
' 2. The two bosses created during the recording of the hidden API Undo object are undone.
' 3. The cut extrusion created after the recording of the hidden API Undo object is undone.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim boolstatus As Boolean
Dim longstatus As Long
Option Explicit
Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2013\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc3 "Part2", False, swUserDecision, longstatus
    Set Part = swApp.ActiveDoc

    Set swModelDocExt = Part.Extension

    Dim myModelView As SldWorks.ModelView
    Set myModelView = Part.ActiveView
    myModelView.FrameState = swWindowState_e.swWindowMaximized
    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -6.92248508634211E-02, 3.92379182115397E-02, 9.87134779060705E-
03, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-8.91172006155176E-02, 0.0314069429482, 0, -4.25302352423542E-02,
6.01966406507166E-03, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    Dim skSegment As SldWorks.SketchSegment
    Set skSegment = Part.SketchManager.CreateCircle(0.009029, 0.03036, 0#, 0.021854, 0.019629, 0#)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    Set skSegment = Part.SketchManager.CreateEllipse(3.06284568434307E-02, 6.19756829649987E-03, 0, 3.09763470298606E-02,
9.97419305453208E-03, 0, 2.86971648691861E-02, 6.37547252792807E-03, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    Set skSegment = Part.SketchManager.CreateEllipse(2.40620641310443E-02, 1.31240684851264E-02, 0, 7.71974468433887E-02,
7.06711158113391E-02, 0, 8.86560440335415E-04, 3.45228945826079E-02, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True

    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    Dim myFeature As SldWorks.Feature
    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False,
1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)
    Part.SelectionManager.EnableContourSelection = False

    ' Start recording the SOLIDWORKS Undo object
    swModelDocExt.StartRecordingUndoObject

    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False,
1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)
    Part.SelectionManager.EnableContourSelection = False
    boolstatus = Part.Extension.SelectByID2("Sketch4", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False,
1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)

```

```
Part.SelectionManager.EnableContourSelection = False
```

```
' End recording the SOLIDWORKS Undo object with name "API Undo" and hide it in the Undo list  
swModelDocExt.FinishRecordingUndoObject2 "API Undo", True
```

```
Part.ClearSelection2 True
```

```
boolstatus = Part.Extension.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
```

```
Set myFeature = Part.FeatureManager.FeatureCut3(True, False, True, 0, 0, 0.00254, 0.00254, False, False, False, False,  
1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, True, False, 0, 0, False)
```

```
Part.SelectionManager.EnableContourSelection = False
```

```
End Sub
```

Create Hidden Undo Object Example (VB.NET)

This example shows how to create an Undo object that is hidden in the SOLIDWORKS Undo list.

```

'-----
' Preconditions: Ensure the part template path exists.
'
' Postconditions:
' 1. A part with four sketches is created.
' 2. One sketch is extruded.
' 3. A hidden Undo object, API Undo, is created with two extrusions.
' 4. One sketch is cut extruded.
' 5. The following items appear in the SOLIDWORKS Undo list in this order:
'   a. Extruded Cut
'   b. (API Undo, hidden from view)
'   c. Base
'
' NOTE: If you select Base in the SOLIDWORKS Undo list:
' 1. The base boss created before the recording of the hidden API Undo object is undone.
' 2. The two bosses created during the recording of the hidden API Undo object are undone.
' 3. The cut extrusion created after the recording of the hidden API Undo object is undone.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swModelView as ModelView
    Dim swSketchManager As SketchManager
    Dim swSketchSegment As SketchSegment
    Dim swFeature As Feature
    Dim swFeatureManager As FeatureManager
    Dim swSelectionManager As SelectionMgr
    Dim status As Boolean
    Dim errors As Integer

    Sub Main()

        swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2013\templates\Part.prtdot", swDwgPaperSizes_e.swDwgPaperASize, 0, 0)
        swApp.ActivateDoc3("Part2.sldprt", False, swRebuildOnActivation_e.swDontRebuildActiveDoc, errors)
        swModel = swApp.ActiveDoc

        swModelView = swModel.ActiveView

        swModelView.FrameState = swWindowState_e.swWindowMaximized

        swModelDocExt = swModel.Extension
        swSketchManager = swModel.SketchManager

        swSketchManager.InsertSketch(True)
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0692248508634211, 0.0392379182115397, 0.00987134779060705, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)

        Dim vSkLines As Object
        vSkLines = swSketchManager.CreateCornerRectangle(-0.0891172006155176, 0.0314069429482, 0, -0.0425302352423542, 0.00601966406507166, 0)
        swModel.ClearSelection2(True)

        swSketchManager.InsertSketch(True)
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)

        swSketchSegment = swSketchManager.CreateCircle(0.009029, 0.03036, 0.0#, 0.021854, 0.019629, 0.0#)
        swModel.ClearSelection2(True)

        swSketchManager.InsertSketch(True)
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)

        swSketchSegment = swSketchManager.CreateEllipse(0.0306284568434307, 0.00619756829649987, 0, 0.0309763470298606, 0.00997419305453208, 0, 0.0286971648691861, 0.00637547252792807, 0)
        swModel.ClearSelection2(True)

        swSketchManager.InsertSketch(True)
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)

        swSketchSegment = swSketchManager.CreateEllipse(0.0240620641310443, 0.0131240684851264, 0, 0.0771974468433887, 0.0706711158113391, 0, 0.000886560440335415, 0.0345228945826079, 0)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)
        swSelectionManager = swModel.SelectionManager
        swSelectionManager.EnableContourSelection = False

        ' Start recording the SOLIDWORKS Undo object
        swModelDocExt.StartRecordingUndoObject()
        status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)
        swSelectionManager.EnableContourSelection = False
        status = swModelDocExt.SelectByID2("Sketch4", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)
        swSelectionManager.EnableContourSelection = False

        ' End recording the SOLIDWORKS Undo object with name "API Undo" and hide it in the Undo list
        swModelDocExt.FinishRecordingUndoObject2("API Undo", True)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Sketch3", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        swFeature = swFeatureManager.FeatureCut3(True, False, True, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, True, False, 0, 0, False)
        swSelectionManager.EnableContourSelection = False

    End Sub

```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Create New Part from Existing Part Using Temporary Body Example (C#)

This example shows how to delete faces from a temporary body and how to create a new part using that temporary body.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\toolbox\braceright.sldprt.
// 2. Verify that the specified part template exists..
//
// Postconditions:
// 1. Creates a new part; the new part has same body as original part
//    but with selected faces deleted.
// 2. Close the new part without saving it.
// 3. Close braceright.sldprt without saving it.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace DeleteFaces5Body2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public object GetFacesWithAttribute(SldWorks swApp, Body2 swBody, AttributeDef swAttDef)
        {
            Face2 swFace = default(Face2);
            Entity swEnt = default(Entity);
            SolidWorks.Interop.sldworks.Attribute swAttCopy = default(SolidWorks.Interop.sldworks.Attribute);
            // Search for faces on temporary body based on copied attributes
            Face2[] swFaceArr = new Face2[1];
            swFace = (Face2)swBody.GetFirstFace();
            while ((null != swFace))
            {
                {
                    swEnt = (Entity)swFace;
                    swAttCopy = null;
                    // Only one instance of attribute on a face should exist
                    swAttCopy = (SolidWorks.Interop.sldworks.Attribute)swEnt.FindAttribute(swAttDef, 0);
                    if ((swAttCopy != null))
                    {
                        swFaceArr[swFaceArr.GetUpperBound(0)] = swFace;
                        Array.Resize(ref swFaceArr, swFaceArr.GetUpperBound(0) + 2);
                    }
                    swFace = (Face2)swFace.GetNextFace();
                }
            }
            Array.Resize(ref swFaceArr, swFaceArr.GetUpperBound(0));
            return swFaceArr;
        }
        public void Main()
        {
            const int CreateVisible = 0;
            const string sAttDefName = "temp_attrib";
            const string sAttRootName = "root_attrib";
            AttributeDef swAttDef = default(AttributeDef);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            PartDoc swPart = default(PartDoc);
            Body2 swBody = default(Body2);
            Body2 swCopyBody = default(Body2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            long nSelCount = 0;
            Face2 swFace = default(Face2);
            Entity swEnt = default(Entity);
            SolidWorks.Interop.sldworks.Attribute[] swAtt = new SolidWorks.Interop.sldworks.Attribute[6];
```

```

        object[] vFaceArr = null;
        PartDoc swNewPart = default(PartDoc);
        ModelDoc2 swNewModel = default(ModelDoc2);
        Feature swFeat = default(Feature);
        object[] vBodies = null;
        bool boolstatus = false;
        int i = 0;
        bool bLocChk = false;
        bool bRet = false;

        swAttDef = (AttributeDef)swApp.DefineAttribute(sAttDefName);
        swModel = (ModelDoc2)swApp.ActiveDoc;
        swModelDocExt = swModel.Extension;
        swSelMgr = (SelectionMgr)swModel.SelectionManager;
        swPart = (PartDoc)swModel;
        bRet = swAttDef.Register();
        //Debug.Assert(bRet);
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.02203398034251, 0.2107859236428, 0.005471558832284, true, 0, null,
0);
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.03651723484872, 0.1911276369938, 0.007226351471076, true, 0, null,
0);
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.01524, 0.1384548315647, 0.004444480215071, true, 0, null, 0);
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1306826750488, 0.0172129316129, 0.006448917397336, true, 0, null,
0);
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1068570742154, 0.01524000000001, 0.00670683128584, true, 0, null,
0);
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.01652926606039, 0.01775444632528, 0.004157527166058, true, 0,
null, 0);
        // Add attribute to selected faces
        nSelCount = swSelMgr.GetSelectedObjectCount2(-1);

        for (i = 1; i <= nSelCount; i++)
        {
            swFace = (Face2)swSelMgr.GetSelectedObject6(i, -1);
            swEnt = (Entity)swFace;
            swAtt[i - 1] = swAttDef.CreateInstance5(swModel, swEnt, sAttRootName + i, CreateVisible,
(int)swInConfigurationOpts_e.swAllConfiguration);

        }
        Object varBodies;
        vBodies = (Object[])swPart.GetBodies2((int)swBodyType_e.swAllBodies, true);
        varBodies = vBodies;
        swBody = (Body2)vBodies[0];
        swCopyBody = (Body2)swBody.Copy();
        // Remove attribute from faces
        for (i = 1; i <= nSelCount; i++)
        {
            bRet = swAtt[i - 1].Delete(true);

        }
        vFaceArr = (Object[])GetFacesWithAttribute(swApp, swCopyBody, swAttDef);
        Object varFaceArr = vFaceArr;
        bRet = swCopyBody.DeleteFaces5(varFaceArr, (int)swHealActionType_e.swHealAction_Shrink,
(int)swLoopProcessOption_e.swLoopProcess_Auto, true, out varBodies, out bLocChk);

        swNewPart = (PartDoc)swApp.NewDocument("C:\\Documents and Settings\\All Users\\Application Data\\SOLIDWORKS\\SOLIDWORKS
2016\\templates\\part.prtdot", 0, 0, 0);
        swNewModel = (ModelDoc2)swNewPart;
        swFeat = (Feature)swNewPart.CreateFeatureFromBody3(swCopyBody, false,
(int)swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck);
    }
    public SldWorks swApp;
}
}

```

Create New Part from Existing Part Using Temporary Body Example (VBA)

This example shows how to delete faces from a temporary body and how to create a new part using that temporary body.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\toolbox\braceright.sldprt.
' 2. Verify that the specified part template exists..
'
' Postconditions:
' 1. Creates a new part; the new part has same body as original part
'    but with selected faces deleted.
' 2. Close the new part without saving it.
' 3. Close braceright.sldprt without saving it.
'-----

```

Option Explicit

Function GetFacesWithAttribute(swApp As SldWorks.SldWorks, swBody As SldWorks.Body2, swAttDef As SldWorks.AttributeDef) As Variant

```

    Dim swFace                As SldWorks.Face2
    Dim swEnt                  As SldWorks.Entity
    Dim swAttCopy              As SldWorks.Attribute
    Dim swFaceArr()            As SldWorks.Face2

    ' Search for faces on temporary body based on copied attributes
    ReDim swFaceArr(0)
    Set swFace = swBody.GetFirstFace

    Do While Not Nothing Is swFace
        Set swEnt = swFace
        Set swAttCopy = Nothing
        ' Only one instance of attribute on a face should exist
        Set swAttCopy = swEnt.FindAttribute(swAttDef, 0)
        If Not swAttCopy Is Nothing Then
            Set swFaceArr(UBound(swFaceArr)) = swFace
            ReDim Preserve swFaceArr(UBound(swFaceArr) + 1)
        End If
        Set swFace = swFace.GetNextFace
    Loop

    Debug.Assert UBound(swFaceArr) >= 1
    ReDim Preserve swFaceArr(UBound(swFaceArr) - 1)
    GetFacesWithAttribute = swFaceArr

```

End Function

```

Sub main()
    ' 1 = invisible
    ' 0 = visible
    Const CreateVisible As Long = 0
    Const sAttDefName As String = "temp_attrib"
    Const sAttRootName As String = "root_attrib"
    Dim swApp As SldWorks.SldWorks
    Dim swAttDef As SldWorks.AttributeDef
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swPart As SldWorks.PartDoc
    Dim swBody As SldWorks.Body2
    Dim swCopyBody As SldWorks.Body2
    Dim swSelMg As SldWorks.SelectionMgr
    Dim nSelCount As Long
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim swAtt() As SldWorks.Attribute

```

```

Dim vFaceArr As Variant
Dim swNewPart As SldWorks.PartDoc
Dim swNewModel As SldWorks.ModelDoc2
Dim swFeat As SldWorks.Feature
Dim swFaultEntity As SldWorks.FaultEntity
Dim vBodies As Variant
Dim boolstatus As Boolean
Dim i As Long
Dim bLocChk As Boolean
Dim bRet As Boolean

Set swApp = Application.SldWorks
Set swAttDef = swApp.DefineAttribute(sAttDefName)
Set swModel = swApp.ActiveDoc
Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager
Set swPart = swModel

bRet = swAttDef.Register: Debug.Assert bRet

boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.02203398034251, 0.2107859236428, 0.005471558832284, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.03651723484872, 0.1911276369938, 0.007226351471076, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.01524, 0.1384548315647, 0.004444480215071, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1306826750488, 0.0172129316129, 0.006448917397336, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1068570742154, 0.015240000000001, 0.00670683128584, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.01652926606039, 0.01775444632528, 0.004157527166058, True, 0, Nothing, 0)
' Add attribute to selected faces
nSelCount = swSelMgr.GetSelectedObjectCount2(-1)
ReDim swAtt(nSelCount)
For i = 1 To nSelCount
    Set swFace = swSelMgr.GetSelectedObject6(i, -1)
    Set swEnt = swFace
    Set swAtt(i - 1) = swAttDef.CreateInstance5(swModel, swEnt, sAttRootName & i, CreateVisible, swAllConfiguration):
Debug.Assert Not swAtt(i - 1) Is Nothing
Next i
vBodies = swPart.GetBodies2(swAllBodies, True)
Set swBody = vBodies(0)
Set swCopyBody = swBody.Copy

' Remove attribute from faces
For i = 1 To nSelCount
    bRet = swAtt(i - 1).Delete(True): Debug.Assert bRet
Next i

vFaceArr = GetFacesWithAttribute(swApp, swCopyBody, swAttDef)
Debug.Assert nSelCount = UBound(vFaceArr) + 1

' Can only delete faces from a temporary body
Debug.Assert swCopyBody.IsTemporaryBody

' Should not assert because it may fail to delete faces or fail local check or both
bRet = swCopyBody.DeleteFaces5(vFaceArr, swHealAction_Shrink, swLoopProcess_Auto, True, vBodies, bLocChk): Debug.Assert bRet:
Debug.Assert bLocChk

Set swNewPart = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS
2016\templates\part.prtdot", 0, 0, 0)
Set swNewModel = swNewPart
Set swFeat = swNewPart.CreateFeatureFromBody3(swCopyBody, False, swCreateFeatureBodyCheck): Debug.Assert Not swFeat Is Nothing

End Sub

```

Create New Part from Existing Part Using Temporary Body Example (VB.NET)

This example shows how to delete faces from a temporary body and how to create a new part using that temporary body.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\toolbox\braceright.sldprt.
' 2. Verify that the specified part template exists..
'
' Postconditions:
' 1. Creates a new part; the new part has same body as original part
'    but with selected faces deleted.
' 2. Close the new part without saving it.
' 3. Close braceright.sldprt without saving it.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
Function GetFacesWithAttribute(ByVal swApp As SldWorks, ByVal swBody As Body2, ByVal swAttDef As AttributeDef) As Object
```

```
    Dim swFace As Face2
    Dim swEnt As Entity
    Dim swAttCopy As SolidWorks.Interop.sldworks.Attribute
    Dim swFaceArr() As Face2

    ' Search for faces on temporary body based on copied attributes
    ReDim swFaceArr(0)
    swFace = swBody.GetFirstFace
    Do While Not Nothing Is swFace
        swEnt = swFace
        swAttCopy = Nothing
        ' Only one instance of attribute on a face should exist
        swAttCopy = swEnt.FindAttribute(swAttDef, 0)
        If Not swAttCopy Is Nothing Then
            swFaceArr(UBound(swFaceArr)) = swFace
            ReDim Preserve swFaceArr(UBound(swFaceArr) + 1)
        End If
        swFace = swFace.GetNextFace
    Loop

    Debug.Assert(UBound(swFaceArr) >= 1)
    ReDim Preserve swFaceArr(UBound(swFaceArr) - 1)
    GetFacesWithAttribute = swFaceArr
```

```
End Function
```

```
Public Sub main()
```

```
    ' 1 = invisible
    ' 0 = visible

    Const CreateVisible As Long = 0
    Const sAttDefName As String = "temp_attrib"
    Const sAttRootName As String = "root_attrib"
    Dim swAttDef As AttributeDef
    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swPart As PartDoc
    Dim swBody As Body2
    Dim swCopyBody As Body2
```

```

Dim swSelMgr As SelectionMgr
Dim nSelCount As Long
Dim swFace As Face2
Dim swEnt As Entity
Dim swAtt() As SolidWorks.Interop.sldworks.Attribute
Dim vFaceArr As Object
Dim swNewPart As PartDoc
Dim swNewModel As ModelDoc2
Dim swFeat As Feature
Dim vBodies As Object
Dim boolstatus As Boolean
Dim i As Long
Dim bLocChk As Boolean
Dim bRet As Boolean

swAttDef = swApp.DefineAttribute(sAttDefName)
swModel = swApp.ActiveDoc
swModelDocExt = swModel.Extension
swSelMgr = swModel.SelectionManager
swPart = swModel

bRet = swAttDef.Register : Debug.Assert(bRet)
0) boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.02203398034251, 0.2107859236428, 0.005471558832284, True, 0, Nothing,
0) boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.03651723484872, 0.1911276369938, 0.007226351471076, True, 0, Nothing,
0) boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.01524, 0.1384548315647, 0.004444480215071, True, 0, Nothing, 0)
0) boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1306826750488, 0.0172129316129, 0.006448917397336, True, 0, Nothing,
0) boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1068570742154, 0.01524000000001, 0.00670683128584, True, 0, Nothing,
0) boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.01652926606039, 0.01775444632528, 0.004157527166058, True, 0, Nothing,
0)

' Add attribute to selected faces
nSelCount = swSelMgr.GetSelectedObjectCount2(-1)
ReDim swAtt(nSelCount)
For i = 1 To nSelCount
    swFace = swSelMgr.GetSelectedObject6(i, -1)
    swEnt = swFace
    swAtt(i - 1) = swAttDef.CreateInstance5(swModel, swEnt, sAttRootName & i, CreateVisible,
swInConfigurationOpts_e.swAllConfiguration) : Debug.Assert(Not swAtt(i - 1) Is Nothing)
Next i

vBodies = swPart.GetBodies2(swBodyType_e.swAllBodies, True)
swBody = vBodies(0)
swCopyBody = swBody.Copy

' Remove attribute from faces
For i = 1 To nSelCount
    bRet = swAtt(i - 1).Delete(True) : Debug.Assert(bRet)
Next i

vFaceArr = GetFacesWithAttribute(swApp, swCopyBody, swAttDef)
Debug.Assert(nSelCount = UBound(vFaceArr) + 1)

' Can only delete faces from a temporary body
Debug.Assert(swCopyBody.IsTemporaryBody)

' Should not assert because it may fail to delete faces or fail local check or both
bRet = swCopyBody.DeleteFaces5(vFaceArr, swHealActionType_e.swHealAction_Shrink, swLoopProcessOption_e.swLoopProcess_Auto,
True, vBodies, bLocChk) : Debug.Assert(bRet) : Debug.Assert(bLocChk)
    swNewPart = swApp.NewDocument("C:\Documents and Settings\All Users\Application Data\SOLIDWORKS\SOLIDWORKS
2016\templates\part.prt", 0, 0, 0)
    swNewModel = swNewPart
    swFeat = swNewPart.CreateFeatureFromBody3(swCopyBody, False, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck) :
Debug.Assert(Not swFeat Is Nothing)

```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Create Non-associative Diameter Dimension Example (VBA)

This example shows how to create a non-associative diameter dimension.

```
'-----
' Preconditions: Open a drawing.
'
' Postconditions:
' 1. Creates a non-associative diameter
'    dimension in the drawing.
' 2. Examine the drawing.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDispDim As SldWorks.DisplayDimension
    Dim nDimLoc(2) As Double
    Dim nCircPt1(2) As Double
    Dim nCircPt2(2) As Double
    Dim nPlaneNorm(2) As Double
    Dim nTextPt(2) As Double
    Dim vDimLoc As Variant
    Dim vCircPt1 As Variant
    Dim vCircPt2 As Variant
    Dim vPlaneNorm As Variant
    Dim vTextPt As Variant
    Dim nDimVal As Double
    Dim nTextHt As Double

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swDraw = swModel

    nDimLoc(0) = 0.1
    nDimLoc(1) = 0.1
    nDimLoc(2) = 0.1
    vDimLoc = nDimLoc

    nCircPt1(0) = 0.64
    nCircPt1(1) = 0.112
    nCircPt1(2) = 0#
    vCircPt1 = nCircPt1

    nCircPt2(0) = 0.125
    nCircPt2(1) = 0.14
    nCircPt2(2) = 0#
    vCircPt2 = nCircPt2

    nPlaneNorm(0) = 0#
    nPlaneNorm(1) = 0#
    nPlaneNorm(2) = 1#
    vPlaneNorm = nPlaneNorm

    nTextPt(2) = 0.2
    nTextPt(2) = 0.2
    nTextPt(2) = 0.2
    vTextPt = nTextPt

    nDimVal = 0.1255
    nTextHt = 0.01

    Set swDispDim = swDraw.CreateDiamDim4(vDimLoc, vCircPt1, vCircPt2, vPlaneNorm, vTextPt, nDimVal, nTextHt)
    swDispDim.Diametric = False

    swModel.GraphicsRedraw2

End Sub
```

Create Normal Cut Feature Example (C#)

This example shows how to create a normal cut feature.

```
//-----
// Preconditions:
// 1. Open:
//    c:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\whatsnew\sheet metal\normal_cut.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates Normal Cut1 in the FeatureManager design tree.
// 2. Edits the feature by specifying a normal cut direction entity.
// 3. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save any changes to it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CreateNormalCut_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private ModelDocExtension swModelDocExt;
        private FeatureManager swFeatMgr;
        private Feature swFeat;
        private SMNormalCutFeatureData2 swNormalCutFeatData;
        private SMNormalCutGroupData grpData;
        private SelectionMgr swSelMgr;
        private Face2 swCutDirectionFace;
        private int errCode;
        private bool boolstatus;
        private object[] varGrpNames;
        private int i;
        private int j;
        private string Name;
        private Face2 Face;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swFeatMgr = swModel.FeatureManager;
            swNormalCutFeatData = (SMNormalCutFeatureData2)swFeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmNormalCut);
            swNormalCutFeatData.AutoPropagation = true;
            swNormalCutFeatData.OptimizeGeometry = true;
            swNormalCutFeatData.NormalCutParameters = (int)swNormalCutParameters_e.swNormalCutExtent;

            // Select the face(s) to make normal

            boolstatus = swModel.Extension.SelectByRay(0.0471296104838217, 0.0142054003862171, -0.00668923799798904, -0.133226865456);

            grpData = (SMNormalCutGroupData)swNormalCutFeatData.CreateGroup(out errCode);

            // Create the normal cut feature

            swFeat = swFeatMgr.CreateFeature(swNormalCutFeatData);

            // Modify the normal cut feature

            swNormalCutFeatData = (SMNormalCutFeatureData2)swFeat.GetDefinition();
            swNormalCutFeatData.AccessSelections(swModel, null);
            varGrpNames = (object[])swNormalCutFeatData.GetGroupNames();

            for (i = 0; i <= varGrpNames.GetUpperBound(0); i++)
            {
                Name = (string)varGrpNames[i];
                Debug.Print("GroupName: " + Name);
                grpData = (SMNormalCutGroupData)swNormalCutFeatData.GetGroupByName(Name);
                object[] varFaces = (object[])grpData.Faces;
                for (j = 0; j <= varFaces.GetUpperBound(0); j++)
                {
                    Face = (Face2)varFaces[j];
                    Debug.Print("Face " + j + " area: " + Face.GetArea());
                }
                Debug.Print("-----");
            }

            //Specify the cut direction

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
```

```

boolstatus = swModel.Extension.SelectByRay(-0.149999999999892, 0.0258556514245498, -0.00463505465353364, 0.3932426685182;
swCutDirectionFace = null;
swCutDirectionFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);

Entity swSelectEnt = (Entity)swCutDirectionFace;
boolstatus = swSelectEnt.Select4(false, null);
swNormalCutFeatData.CutDirection = swCutDirectionFace;

bool isModified = swFeat.ModifyDefinition(swNormalCutFeatData, swModel, null);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

}

```

Create Note Example (VBA)

This example shows how to compose a two-line note and insert the note on the current drawing sheet.

```
'-----  
' Preconditions: Open a drawing.  
'  
' Postconditions:  
' 1. Verifies that a drawing is open.  
' 2. Composes and inserts a note on the  
'    drawing sheet.  
' 3. Examine the graphics area.  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim str As String  
  
    ' Constant enumerators  
    Const swDocPART = 1  
    Const swDocASSEMBLY = 2  
    Const swDocDRAWING = 3  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    If swModel Is Nothing Then  
        ' If no model currently loaded, then exit  
        Exit Sub  
    End If  
  
    ' Determine the document type  
    ' If the document is not a drawing, then send a message to the user  
    If (swModel.GetType <> swDocDRAWING) Then  
        swApp.SendMsgToUser ("Macro only used for drawings")  
        Exit Sub  
    End If  
  
    ' Compose text string with carriage return  
    str = "Fred and " + Chr(10) + "Ethel"  
  
    ' Insert note at (.09,.08) on the sheet  
    swModel.CreateText str, 0.09, 0.08, 0, 0.01, 0  
  
End Sub
```

Create On-surface Spline Example (C#)

This example shows how to create a spline constrained to a surface.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\cstick.sldprt.
//
// Postconditions:
// 1. Creates a 3D sketch of a spline on a face.
// 2. Inspect the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateSurfSpline_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            Face2 swCurFace = default(Face2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDoc2 Part = default(ModelDoc2);
            SketchSegment skSegment = default(SketchSegment);
            object pointArray = null;
            double[] points = new double[12];
            object[] surfs = new object[1];
            Surface[] surfaceArr = new Surface[1];
            int selType = 0;
            int surfaceArrUbound = 0;
            object statuses = null;
            bool boolstatus = false;

            Part = (ModelDoc2)swApp.ActiveDoc;

            points[0] = -0.0334270787209949;
            points[1] = 0.0223913424029847;
            points[2] = 0.053671471463652;
            points[3] = 0.0153063989576147;
            points[4] = 0.0478899892864157;
            points[5] = 0.0250019220828396;
            points[6] = 0.0511644715447442;
            points[7] = 0.0272060072570875;
            points[8] = 0.00578476387745854;
            points[9] = 0.00259263831071606;
            points[10] = 0.0262831648993199;
            points[11] = -0.053206707614433;
            pointArray = points;

            boolstatus = Part.Extension.SelectByRay(0.030303902514845, 0.0286262382667246, 0.0385109058888133, -0.577381545199981, -0);

            swSelMgr = (SelectionMgr)Part.SelectionManager;

            selType = swSelMgr.GetSelectedObjectType3(1, -1);
            surfaceArrUbound = -1;
            if (selType == (int)swSelectType_e.swSelFACES)
            {
                swCurFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
                surfaceArrUbound = surfaceArrUbound + 1;
                surfaceArr[surfaceArrUbound] = (Surface)swCurFace.GetSurface();
            }

            surfs = surfaceArr;

            Dispatcher[] dispArray = null;
            dispArray = ObjectArrayToDispatcherArray(surfaceArr);

            //Set the Direction parameter to an array of null DispatchWrappers
            //to use the view vector of the current view to project the points in pointArray
            //onto the surface in dispArray
            Dispatcher dirArray = default(Dispatcher);
            dirArray = new Dispatcher(null);

            skSegment = (SketchSegment)Part.SketchManager.CreateSpline3((pointArray), dispArray, dirArray, true, out statuses);
            Part.SketchManager.InsertSketch(true);
        }
        public Dispatcher[] ObjectArrayToDispatcherArray(Surface[] Objects)
        {
            int ArraySize = 0;
            ArraySize = Objects.GetUpperBound(0);
            Dispatcher[] d = new Dispatcher[ArraySize + 1];
            int ArrayIndex = 0;
```

```
        for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
        {
            d[ArrayIndex] = new DispatcherWrapper(Objects[ArrayIndex]);
        }
        return d;
    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;
}
}
```

Create On-surface Spline Example (VBA)

This example shows how to create a spline constrained to a surface.

 ' Preconditions: Open *public_documents\samples\tutorial\api\cstick.sldprt*.
 '

' Postconditions:
 ' 1. Creates a 3D sketch of a spline on a surface.
 ' 2. Inspect the graphics area.
 '

' NOTE: Because the model is used elsewhere, do not save changes.
 '-----

```
Dim swApp As SldWorks.SldWorks
Dim swCurFace As SldWorks.Face2
Dim swSelMgr As SldWorks.SelectionMgr
Dim Part As SldWorks.ModelDoc2
Dim skSegment As SldWorks.SketchSegment
Dim pointArray As Variant
Dim points() As Double
Dim surfs As Variant
Dim dirs As Variant
Dim surfaceArr() As SldWorks.Surface
Dim selType As Long
Dim surfaceArrUbound As Long
Dim statuses As Variant
Dim boolstatus As Boolean
```

```
Option Explicit
Sub main()
```

```
    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
```

```
    ReDim points(0 To 11) As Double
    points(0) = -3.34270787209949E-02
    points(1) = 2.23913424029847E-02
    points(2) = 0.053671471463652
    points(3) = 1.53063989576147E-02
    points(4) = 4.78899892864157E-02
    points(5) = 2.50019220828396E-02
    points(6) = 5.11644715447442E-02
    points(7) = 2.72060072570875E-02
    points(8) = 5.78476387745854E-03
    points(9) = 2.59263831071606E-03
    points(10) = 2.62831648993199E-02
    points(11) = -0.053206707614433
    pointArray = points
```

```
    boolstatus = Part.Extension.SelectByRay(0.030303902514845, 2.86262382667246E-02, 3.85109058888133E-02, -0.577381545199981,
    -0.577287712085548, -0.577381545199979, 1.78932209693826E-03, 2, False, 0, 0)
```

```
    Set swSelMgr = Part.SelectionManager
```

```
    selType = swSelMgr.GetSelectedObjectType3(1, -1)
    surfaceArrUbound = -1
    If selType = swSelFACES Then
        Set swCurFace = swSelMgr.GetSelectedObject6(1, -1)
        surfaceArrUbound = surfaceArrUbound + 1
        ReDim Preserve surfaceArr(0 To surfaceArrUbound) As Surface
        Set surfaceArr(surfaceArrUbound) = swCurFace.GetSurface()
    End If
```

```
    surfs = surfaceArr
```

```
    'Set Direction parameter to Nothing to use the view vector of the current view
    'to project the points in the pointArray onto the surface in the surfs array
    Set skSegment = Part.SketchManager.CreateSpline3((pointArray), surfs, Nothing, True, statuses)
    Part.SketchManager.InsertSketch True
```

```
End Sub
```

Create On-surface Spline Example (VB.NET)

This example shows how to create a spline constrained to a surface.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\cstick.sldprt.
'
' Postconditions:
' 1. Creates a 3D sketch of a spline on a face.
' 2. Inspect the graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices

```

```

Partial Class SolidWorksMacro

```

```

    Dim swCurFace As Face2
    Dim swSelMgr As SelectionMgr
    Dim Part As ModelDoc2
    Dim skSegment As SketchSegment
    Dim pointArray As Object
    Dim points(11) As Double
    Dim surfs(0) As Object
    Dim surfaceArr(0) As Surface
    Dim selType As Integer
    Dim surfaceArrUbound As Integer
    Dim statuses(3) As Boolean
    Dim boolstatus As Boolean

    Sub main()

        Part = swApp.ActiveDoc

        points(0) = -0.0334270787209949
        points(1) = 0.0223913424029847
        points(2) = 0.053671471463652
        points(3) = 0.0153063989576147
        points(4) = 0.0478899892864157
        points(5) = 0.0250019220828396
        points(6) = 0.0511644715447442
        points(7) = 0.0272060072570875
        points(8) = 0.00578476387745854
        points(9) = 0.00259263831071606
        points(10) = 0.0262831648993199
        points(11) = -0.053206707614433
        pointArray = points

        boolstatus = Part.Extension.SelectByRay(0.030303902514845, 0.0286262382667246, 0.0385109058888133, -0.577381545199981, -0.577381545199981)

        swSelMgr = Part.SelectionManager

        selType = swSelMgr.GetSelectedObjectType3(1, -1)
        surfaceArrUbound = -1
        If selType = swSelectType_e.swSelFACES Then
            swCurFace = swSelMgr.GetSelectedObject6(1, -1)
            surfaceArrUbound = surfaceArrUbound + 1
            surfaceArr(surfaceArrUbound) = swCurFace.GetSurface()
        End If

        surfs = surfaceArr

        Dim dispArray() As DispatcherWrapper
        dispArray = ObjectArrayToDispatcherWrapperArray(surfaceArr)

        'Set the Direction parameter to an array of null DispatcherWrappers
        'to use the view vector of the current view to project the points in pointArray
        'onto the surface in dispArray
        Dim dirArray As DispatcherWrapper
        dirArray = New DispatcherWrapper(Nothing)
        skSegment = Part.SketchManager.CreateSpline3((pointArray), dispArray, dirArray, True, statuses)
        Part.SketchManager.InsertSketch(True)

    End Sub

    Function ObjectArrayToDispatcherWrapperArray(ByVal Objects As Surface()) As DispatcherWrapper()
        Dim ArraySize As Integer
        ArraySize = Objects.GetUpperBound(0)
        Dim d(ArraySize) As DispatcherWrapper
        Dim ArrayIndex As Integer
        For ArrayIndex = 0 To ArraySize
            d(ArrayIndex) = New DispatcherWrapper(Objects(ArrayIndex))
        Next
        Return d
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.

```

```
''' </summary>  
Public swApp As Sldworks
```

```
End Class
```

Create Ordinate Dimensions Example (C#)

This example shows how to create ordinate dimensions in a drawing.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\2012-sm.slddrw.
// 2. Click Tools > Options > Document Properties, expand Dimensions,
// click Ordinate, change Base ordinate dimension standard to DIN,
// and click OK.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Creates ordinate dimensions.
// 2. Click a blank area in the drawing.
// 3. Examine the base ordinate dimension in the drawing and the
// Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            bool status = false;
            int errors = 0;
            bool useDoc = false;
            double arrowSize = 0;

            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            swDrawingDoc = (DrawingDoc)swModelDoc;
            swModelDocExt = (ModelDocExtension)swModelDoc.Extension;
            status = swDrawingDoc.ActivateView("Drawing View1");

            //Create ordinate dimension
            status = swModelDocExt.SelectByID2("", "VERTEX", 0.0876609155372049, 0.255953076919585, -499.97349537912, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", 0.0954286078448972, 0.256322967029476, -499.97349537912, true, 0, null, 0);
            errors = swModelDocExt.AddOrdinateDimension((int)swAddOrdinateDims_e.swHorizontalOrdinate, 0.094688827625117, 0.272968021978022, 0);

            //Add an ordinate dimension
            swModelDoc.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("", "VERTEX", 0.101346849603139, 0.257062747249256, -499.97349537912, false, 0, null, 0);
            swModelDoc.ClearSelection2(true);
            swModelDoc.SetPickMode();

            //Change the diameter of the circle of the base ordinate dimension
            status = swModelDocExt.SelectByID2("D1@Sketch3@2012-sm.SLDDRW", "DIMENSION", 0.0878551078448972, 0.275113384615385, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModelDoc.SelectionManager;
            swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, -1);
            swDisplayDimension.SetOrdinateDimensionArrowSize(false, 0.00288);
            swDisplayDimension.GetOrdinateDimensionArrowSize(out useDoc, out arrowSize);
            Debug.Print("Base ordinate dimension diameter of circle for arrow: " + arrowSize * 1000 + "mm");

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Ordinate Dimensions Example (VBA)

This example shows how to create ordinate dimensions in a drawing.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\2012-sm.slddrw.
' 2. Click Tools > Options > Document Properties, expand Dimensions,
'    click Ordinate, change Base ordinate dimension standard to DIN,
'    and click OK.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates ordinate dimensions.
' 2. Click a blank area in the drawing.
' 3. Examine the base ordinate dimension in the drawing and the
'    Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim status As Boolean
Dim errors As Long
Dim useDoc As Boolean
Dim arrowSize As Double

Sub main()

    Set swApp = Application.SldWorks
    Set swModelDoc = swApp.ActiveDoc
    Set swDrawingDoc = swModelDoc
    Set swModelDocExt = swModelDoc.Extension
    status = swDrawingDoc.ActivateView("Drawing View1")

    'Create ordinate dimension
    status = swModelDocExt.SelectByID("", "VERTEX", 8.76609155372049E-02, 0.255953076919585, -499.97349537912, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID("", "VERTEX", 9.54286078448972E-02, 0.256322967029476, -499.97349537912, True, 0, Nothing, 0)
    errors = swModelDocExt.AddOrdinateDimension(swAddOrdinateDims_e.swHorizontalOrdinate, 0.094688827625117, 0.272968021978022, 0)

    'Add an ordinate dimension
    swModelDoc.ClearSelection2 True
    status = swModelDocExt.SelectByID("", "VERTEX", 0.101346849603139, 0.257062747249256, -499.97349537912, False, 0, Nothing, 0)
    swModelDoc.ClearSelection2 True
    swModelDoc.SetPickMode

    'Change the diameter of the circle of the base ordinate dimension
    status = swModelDocExt.SelectByID("D1@Sketch3@2012-sm.SLDDRW", "DIMENSION", 8.78551078448972E-02, 0.275113384615385, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModelDoc.SelectionManager
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
    swDisplayDimension.SetOrdinateDimensionArrowSize False, 0.00288
    swDisplayDimension.GetOrdinateDimensionArrowSize useDoc, arrowSize
    Debug.Print "Base ordinate dimension diameter of circle for arrow: " & arrowSize * 1000 & "mm"

End Sub
```

Create Ordinate Dimensions Example_VBNET (Topic)

SOLIDWORKS API Help

Create Ordinate Dimensions Example (VB.NET)

This example shows how to create ordinate dimensions in a drawing.

```
-----
Preconditions:
1. Open public_documents\samples\tutorial\api\2012-sm.slddrw.
2. Click Tools > Options > Document Properties, expand Dimensions,
   click Ordinate, change Base ordinate dimension standard to DIN,
   and click OK.
3. Open the Immediate window.

Postconditions:
1. Creates ordinate dimensions.
2. Click a blank area in the drawing.
3. Examine the base ordinate dimension in the drawing and the
   Immediate window.

NOTE: Because the drawing is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModelDoc As ModelDoc2
        Dim swDrawingDoc As DrawingDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension
        Dim status As Boolean
        Dim errors As Integer
        Dim useDoc As Boolean
        Dim arrowSize As Double

        swModelDoc = swApp.ActiveDoc
        swDrawingDoc = swModelDoc
        swModelDocExt = swModelDoc.Extension
        status = swDrawingDoc.ActivateView("Drawing View1")

        'Create ordinate dimension
        status = swModelDocExt.SelectByID("", "VERTEX", 0.0876609155372049, 0.255953076919585, -499.97349537912, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("", "VERTEX", 0.0954286078448972, 0.256322967029476, -499.97349537912, True, 0, Nothing, 0)
        errors = swModelDocExt.AddOrdinateDimension(swAddOrdinateDims_e.swHorizontalOrdinate, 0.094688827625117, 0.272968021978022, 0)

        'Add an ordinate dimension
        swModelDoc.ClearSelection2(True)
        status = swModelDocExt.SelectByID("", "VERTEX", 0.101346849603139, 0.257062747249256, -499.97349537912, False, 0, Nothing, 0)
        swModelDoc.ClearSelection2(True)
        swModelDoc.SetPickMode()

        'Change the diameter of the circle of the base ordinate dimension
        status = swModelDocExt.SelectByID("D1@Sketch3@2012-sm.SLDDRW", "DIMENSION", 0.0878551078448972, 0.275113384615385, 0, False, 0, Nothing, 0)
        swSelMgr = swModelDoc.SelectionManager
        swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
        swDisplayDimension.SetOrdinateDimensionArrowSize(False, 0.00288)
        swDisplayDimension.GetOrdinateDimensionArrowSize(useDoc, arrowSize)
        Debug.Print("Base ordinate dimension diameter of circle for arrow: " & arrowSize * 1000 & "mm")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Create Parabola Example (C#)

This example shows how to create a parabola and get its corresponding equation.

```
//-----
// Preconditions: Verify that the specified template exists.
//
// Postconditions:
// 1. Inserts a sketch.
// 2. Creates a parabola.
// 3. Inspect the Immediate window for the parabolic equation.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Windows.Forms;
namespace CreateParabola_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        const double pi = 3.14159265;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0,
0);

            if (swModel == null)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int)swMessageBoxIcon_e.swMbWarning,
(int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            int modelType = 0;
            modelType = swModel.GetType();

            if (modelType != (int)swDocumentTypes_e.swDocPART)
            {
                swApp.SendMsgToUser2("A part document must be active.", (int)swMessageBoxIcon_e.swMbWarning,
(int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            //Select a plane on which to sketch
            if (SelectPlane(swModel) == false)
            {
                MessageBox.Show("Could not select plane.");
                return;
            }

            //Get point data
            SketchPoint pFocal = default(SketchPoint);
            SketchPoint pApex = default(SketchPoint);
            SketchPoint pStart = default(SketchPoint);
            SketchPoint pEnd = default(SketchPoint);
            SketchManager swSkMgr = default(SketchManager);
            swSkMgr = swModel.SketchManager;

            SelectionMgr swSelMgr = default(SelectionMgr);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            Sketch swSketch = default(Sketch);
            swSkMgr.InsertSketch(true);
            swSketch = swSkMgr.ActiveSketch;

            // Focal point
            pFocal = swSkMgr.CreatePoint(0, -0.025930732990048, 0);
            // Apex point
            pApex = swSkMgr.CreatePoint(0.0110754938634727, -0.0485199777778575, 0);
            // Start point
            pStart = swSkMgr.CreatePoint(0.057136404168939, 0.0869770346454566, 0);
            // End point
            pEnd = swSkMgr.CreatePoint(-0.120690397734139, -0.00465528735997846, 0);

            object[] vPoint = null;

            // Make sure a sketch is active
            if (swSketch == null)
            {
                MessageBox.Show("Please sketch a focal point, apex point, start point, and end point.");
            }
        }
    }
}
```

```

        return;
    }

    vPoint = (object[])swSketch.GetSketchPoints2();

    // Make sure the sketch has the necessary points
    if (vPoint.GetUpperBound(0) <= 2)
    {
        MessageBox.Show("You did not sketch enough points to define the parabola. Please sketch a focal point, apex point, start point, and end point.");
        return;
    }

    SketchParabola SkParabola = default(SketchParabola);
    SkParabola = (SketchParabola)swModel.SketchManager.CreateParabola(pFocal.X, pFocal.Y, 0, pApex.X, pApex.Y, 0, pStart.X, pStart.Y, 0, pEnd.X, pEnd.Y, 0);

    swModel.ViewZoomtofit2();

    //Extract information about the parabola
    pApex = (SketchPoint)SkParabola.GetApexPoint2();
    pStart = (SketchPoint)SkParabola.GetStartPoint2();
    pEnd = (SketchPoint)SkParabola.GetEndPoint2();
    pFocal = (SketchPoint)SkParabola.GetFocalPoint2();

    Debug.Print("    Apex Point = (" + pApex.X * 1000 + ", " + pApex.Y * 1000 + ", " + pApex.Z * 1000 + ") mm");
    Debug.Print("    Start Point = (" + pStart.X * 1000 + ", " + pStart.Y * 1000 + ", " + pStart.Z * 1000 + ") mm");
    Debug.Print("    End Point = (" + pEnd.X * 1000 + ", " + pEnd.Y * 1000 + ", " + pEnd.Z * 1000 + ") mm");
    Debug.Print("    Focal Point = (" + pFocal.X * 1000 + ", " + pFocal.Y * 1000 + ", " + pFocal.Z * 1000 + ") mm");

    // Define point parameters
    if (!(pFocal.Z == 0) | !(pApex.Z == 0) | !(pStart.Z == 0) | !(pEnd.Z == 0))
    {
        MessageBox.Show("Need a 2D sketch.");
        return;
    }

    // Algebraic equation of parabola
    double h = 0;
    double p = 0;
    double k = 0;

    h = pApex.X;
    k = pApex.Y;

    // Correct anomalies when the parabola is aligned with the x and y axes
    if (pFocal.Y == pApex.Y)
    {
        if (pFocal.X > pApex.X)
        {
            p = Math.Sqrt(Math.Pow((pFocal.Y - pApex.Y), 2) + Math.Pow((pFocal.X - pApex.X), 2));
        }
        else
        {
            p = -(Math.Sqrt(Math.Pow((pFocal.Y - pApex.Y), 2) + Math.Pow((pFocal.X - pApex.X), 2)));
        }
    }

    if (pFocal.X == pApex.X)
    {
        if (pFocal.Y > pApex.Y)
        {
            p = Math.Sqrt(Math.Pow((pFocal.Y - pApex.Y), 2) + Math.Pow((pFocal.X - pApex.X), 2));
        }
        else
        {
            p = -(Math.Sqrt(Math.Pow((pFocal.Y - pApex.Y), 2) + Math.Pow((pFocal.X - pApex.X), 2)));
        }
    }

    if (pFocal.X != pApex.X & pFocal.Y != pApex.Y)
    {
        p = (Math.Sqrt(Math.Pow((pFocal.Y - pApex.Y), 2) + Math.Pow((pFocal.X - pApex.X), 2)));
    }

    double a = 0;
    double b = 0;
    double c = 0;
    double c1 = 0;
    double c2 = 0;
    double c3 = 0;
    double c4 = 0;
    double c5 = 0;
    double c6 = 0;
    double theta = 0;

    // Obtain the correct value for theta as the parabola rotates
    if (pFocal.X != pApex.X & pFocal.Y != pApex.Y)
    {
        a = 1 / (4 * p);
        b = -k / (2 * p);
        c = (k * k / (4 * p)) + h;

        // Theta in first quadrant
        if (pFocal.Y > pApex.Y & pFocal.X > pApex.X)

```

```

        {
            theta = Math.Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X));
        }

        // Theta in second quadrant
        if (pFocal.Y > pApex.Y & pFocal.X < pApex.X)
        {
            theta = (Math.Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + pi;
        }

        // Theta in the third quadrant
        if (pFocal.Y < pApex.Y & pFocal.X < pApex.X)
        {
            theta = (Math.Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + pi;
        }

        // Theta in the fourth quadrant
        if (pFocal.Y < pApex.Y & pFocal.X > pApex.X)
        {
            theta = (Math.Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + (2 * pi);
        }

        c1 = Math.Round(a * Math.Pow((Math.Sin(theta)), 2), 2);
        c2 = Math.Round(-a * Math.Sin(2 * theta), 2);
        c3 = Math.Round(a * Math.Pow((Math.Cos(theta)), 2), 2);
        c4 = Math.Round((-b * Math.Sin(theta)) - Math.Cos(theta), 2);
        c5 = Math.Round((b * Math.Cos(theta)) - Math.Sin(theta), 2);
        c6 = Math.Round(c, 2);

        Debug.Print("Equation of the parabola: " + c1 + "x^2 + " + c2 + "xy + " + c3 + "y^2 + " + c4 + "x + " + c5 + "y + "
+ c6 + " = 0");
    }

    // If the parabola has a vertical axis of symmetry
    if (pFocal.X == pApex.X)
    {
        a = 1 / (4 * p);
        b = -h / (2 * p);
        c = (Math.Pow(h, 2) / (4 * p)) + k;
        c1 = Math.Round(a, 2);
        c4 = Math.Round(b, 2);
        c6 = Math.Round(c, 2);
        Debug.Print("Equation of the parabola: y = " + c1 + "x^2 + " + c4 + "x + " + c6);
    }

    // If the parabola has a horizontal axis of symmetry
    if (pFocal.Y == pApex.Y)
    {
        a = 1 / (4 * p);
        b = -k / (2 * p);
        c = (k * k / (4 * p)) + h;
        c3 = Math.Round(a, 2);
        c5 = Math.Round(b, 2);
        c6 = Math.Round(c, 2);
        Debug.Print("Equation of the parabola: x = " + c3 + "y^2 + " + c5 + "y + " + c6);
    }

    swSkMgr.InsertSketch(true);
}

public bool SelectPlane(ModelDoc2 Plane)
{
    bool functionReturnValue = false;

    Feature featureTemp = default(Feature);
    featureTemp = (Feature)Plane.FirstFeature();

    while ((featureTemp != null))
    {
        string sFeatureName = null;
        sFeatureName = featureTemp.GetTypeName2();

        if (sFeatureName == "RefPlane")
        {
            featureTemp.Select2(true, 0);
            functionReturnValue = true;
            return functionReturnValue;
        }

        featureTemp = (Feature)featureTemp.GetNextFeature();
    }

    return functionReturnValue;
}

public SldWorks swApp;
}

```


Create Parabola Example (VBA)

This example shows how to create a parabola and get its corresponding equation.

 ' Preconditions: Verify that the specified template exists.
 '

' Postconditions:
 ' 1. Inserts a sketch.
 ' 2. Creates a parabola.
 ' 3. Inspect the Immediate window for the parabolic equation.

Const pi As Double = 3.14159265

Option Explicit

Sub main()

Dim swApp As SldWorks.SldWorks
 Dim swModel As SldWorks.ModelDoc2

Set swApp = Application.SldWorks
 Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)

If swModel.Is Nothing Then
 swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
 Exit Sub
End If

Dim modelType As Long
 modelType = swModel.GetType

If modelType <> SwConst.swDocPART Then
 swApp.SendMsgToUser2 "A part document must be active.", swMbWarning, swMbOk
 Exit Sub
End If

'Select a plane on which to sketch
 If SelectPlane(swModel) = False Then
 MsgBox "Could not select plane."
 Exit Sub
End If

'Get point data
 Dim pFocal As SldWorks.SketchPoint
 Dim pApex As SldWorks.SketchPoint
 Dim pStart As SldWorks.SketchPoint
 Dim pEnd As SldWorks.SketchPoint
 Dim swSkMgr As SldWorks.SketchManager
 Set swSkMgr = swModel.SketchManager

Dim swSelMgr As SldWorks.SelectionMgr
 Set swSelMgr = swModel.SelectionManager

Dim swSketch As Sketch
 swSkMgr.InsertSketch True
 Set swSketch = swSkMgr.ActiveSketch

' Focal point
 Set pFocal = swSkMgr.CreatePoint(0, -0.025930732990048, 0)
 ' Apex point
 Set pApex = swSkMgr.CreatePoint(1.10754938634727E-02, -4.85199777778575E-02, 0)
 ' Start point
 Set pStart = swSkMgr.CreatePoint(0.057136404168939, 8.69770346454566E-02, 0)
 ' End point
 Set pEnd = swSkMgr.CreatePoint(-0.120690397734139, -4.65528735997846E-03, 0)

Dim vPoint As Variant

' Make sure a sketch is active
 If swSketch.Is Nothing Then
 MsgBox "Please sketch a focal point, apex point, start point, and end point."
 Exit Sub
End If

vPoint = swSketch.GetSketchPoints2

' Make sure the sketch has the necessary points
 If UBound(vPoint) <= 2 Then
 MsgBox " You did not sketch enough points to define the parabola." & vbCrLf & " Please sketch
 a focal point, apex point, start point, and end point."
 Exit Sub
End If

```

Dim SkParabola As SldWorks.SketchParabola
Set SkParabola = swModel.SketchManager.CreateParabola(pFocal.X, pFocal.Y, 0, pApex.X, pApex.Y, 0, pStart.X, pStart.Y, 0,
pEnd.X, pEnd.Y, 0)

```

```

swModel.ViewZoomtofit2

```

```

'Extract information about the parabola
Set pApex = SkParabola.GetApexPoint2
Set pStart = SkParabola.GetStartPoint2
Set pEnd = SkParabola.GetEndPoint2
Set pFocal = SkParabola.GetFocalPoint2

```

```

Debug.Print "      Apex Point = (" & pApex.X * 1000 & ", " & pApex.Y * 1000 & ", " & pApex.Z * 1000 & ") mm"
Debug.Print "      Start Point = (" & pStart.X * 1000 & ", " & pStart.Y * 1000 & ", " & pStart.Z * 1000 & ") mm"
Debug.Print "      End Point = (" & pEnd.X * 1000 & ", " & pEnd.Y * 1000 & ", " & pEnd.Z * 1000 & ") mm"
Debug.Print "      Focal Point = (" & pFocal.X * 1000 & ", " & pFocal.Y * 1000 & ", " & pFocal.Z * 1000 & ") mm"

```

```

' Define point parameters
If Not pFocal.Z = 0 Or Not pApex.Z = 0 Or Not pStart.Z = 0 Or Not pEnd.Z = 0 Then
    MsgBox "Need a 2D sketch."
    Exit Sub
End If

```

```

' Algebraic equation of parabola
Dim h As Double
Dim p As Double
Dim k As Double

```

```

h = pApex.X
k = pApex.Y

```

```

' Correct anomalies when the parabola is aligned with the x and y axes
If pFocal.Y = pApex.Y Then
    If pFocal.X > pApex.X Then
        p = Sqr((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2)
    Else
        p = -(Sqr((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2))
    End If
End If

```

```

If pFocal.X = pApex.X Then
    If pFocal.Y > pApex.Y Then
        p = Sqr((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2)
    Else
        p = -(Sqr((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2))
    End If
End If

```

```

If pFocal.X <> pApex.X And pFocal.Y <> pApex.Y Then
    p = (Sqr((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2))
End If

```

```

Dim a As Double
Dim b As Double
Dim c As Double
Dim c1 As Double
Dim c2 As Double
Dim c3 As Double
Dim c4 As Double
Dim c5 As Double
Dim c6 As Double
Dim theta As Double

```

```

' Obtain the correct value for theta as the parabola rotates
If pFocal.X <> pApex.X And pFocal.Y <> pApex.Y Then
    a = 1 / (4 * p)
    b = -k / (2 * p)
    c = (k * k / (4 * p)) + h

```

```

' Theta in first quadrant
If pFocal.Y > pApex.Y And pFocal.X > pApex.X Then
    theta = Atn((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))
End If

```

```

' Theta in second quadrant
If pFocal.Y > pApex.Y And pFocal.X < pApex.X Then
    theta = (Atn((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + pi
End If

```

```

' Theta in the third quadrant
If pFocal.Y < pApex.Y And pFocal.X < pApex.X Then
    theta = (Atn((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + pi
End If

```

```

' Theta in the fourth quadrant
If pFocal.Y < pApex.Y And pFocal.X > pApex.X Then
    theta = (Atn((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + (2 * pi)

```

```

End If

c1 = Round(a * (Sin(theta)) ^ 2, 2)
c2 = Round(-a * Sin(2 * theta), 2)
c3 = Round(a * (Cos(theta)) ^ 2, 2)
c4 = Round((-b * Sin(theta)) - Cos(theta), 2)
c5 = Round((b * Cos(theta)) - Sin(theta), 2)
c6 = Round(c, 2)

Debug.Print "Equation of the parabola: " & c1 & "x^2 + " & c2 & "xy + " & c3 & "y^2 + " & c4 & "x + " & c5 & "y + " & c6 &
" = 0"

End If

' If the parabola has a vertical axis of symmetry
If pFocal.X = pApex.X Then
    a = 1 / (4 * p)
    b = -h / (2 * p)
    c = (h ^ 2 / (4 * p)) + k
    c1 = Round(a, 2)
    c4 = Round(b, 2)
    c6 = Round(c, 2)
    Debug.Print "Equation of the parabola: y = " & c1 & "x^2 + " & c4 & "x + " & c6
End If

' If the parabola has a horizontal axis of symmetry
If pFocal.Y = pApex.Y Then
    a = 1 / (4 * p)
    b = -k / (2 * p)
    c = (k * k / (4 * p)) + h
    c3 = Round(a, 2)
    c5 = Round(b, 2)
    c6 = Round(c, 2)
    Debug.Print "Equation of the parabola: x = " & c3 & "y^2 + " & c5 & "y + " & c6
End If

swSkMgr.InsertSketch True
End Sub

Public Function SelectPlane(Plane As SldWorks.ModelDoc2) As Boolean
    Dim featureTemp As Feature
    Set featureTemp = Plane.FirstFeature

    While Not featureTemp Is Nothing
        Dim sFeatureName As String
        sFeatureName = featureTemp.GetTypeName2

        If sFeatureName = "RefPlane" Then
            featureTemp.Select2 True, 0
            SelectPlane = True
            Exit Function
        End If

        Set featureTemp = featureTemp.GetNextFeature
    End While

Wend
End Function

```

Create Parabola Example (VB.NET)

This example shows how to create a parabola and get its corresponding equation.

```

'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Inserts a sketch.
' 2. Creates a parabola.
' 3. Inspect the Immediate window for the parabolic equation.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Math
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Const pi As Double = 3.14159265
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)

```

```
        If swModel Is Nothing Then
            swApp.SendMsgToUser2("A part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)
            Exit Sub
        End If

```

```
        Dim modelType As Integer
        modelType = swModel.GetType

```

```
        If modelType <> swDocumentTypes_e.swDocPART Then
            swApp.SendMsgToUser2("A part document must be active.", swMessageBoxIcon_e.swMbWarning, swMessageBoxBtn_e.swMbOk)
            Exit Sub
        End If

```

```
        'Select a plane on which to sketch
        If SelectPlane(swModel) = False Then
            MsgBox("Could not select plane.")
            Exit Sub
        End If

```

```
        'Get point data
        Dim pFocal As SketchPoint
        Dim pApex As SketchPoint
        Dim pStart As SketchPoint
        Dim pEnd As SketchPoint
        Dim swSkMgr As SketchManager
        swSkMgr = swModel.SketchManager

```

```
        Dim swSelMgr As SelectionMgr
        swSelMgr = swModel.SelectionManager

```

```
        Dim swSketch As Sketch
        swSkMgr.InsertSketch(True)
        swSketch = swSkMgr.ActiveSketch

```

```
        ' Focal point
        pFocal = swSkMgr.CreatePoint(0, -0.025930732990048, 0)
        ' Apex point
        pApex = swSkMgr.CreatePoint(0.0110754938634727, -0.0485199777778575, 0)
        ' Start point
        pStart = swSkMgr.CreatePoint(0.057136404168939, 0.0869770346454566, 0)
        ' End point
        pEnd = swSkMgr.CreatePoint(-0.120690397734139, -0.00465528735997846, 0)

```

```
        Dim vPoint As Object

```

```
        ' Make sure a sketch is active
        If swSketch Is Nothing Then
            MsgBox("Please sketch a focal point, apex point, start point, and end point.")
            Exit Sub
        End If

```

```
        vPoint = swSketch.GetSketchPoints2

```

```
        ' Make sure the sketch has the necessary points
        If UBound(vPoint) <= 2 Then
            MsgBox("You did not sketch enough points to define the parabola." & vbNewLine & " Please sketch a focal point, apex point, start point, and end point.")
            Exit Sub
        End If

```

```

Dim SkParabola As SketchParabola
SkParabola = swModel.SketchManager.CreateParabola(pFocal.X, pFocal.Y, 0, pApex.X, pApex.Y, 0, pStart.X, pStart.Y, 0,
pEnd.X, pEnd.Y, 0)

swModel.ViewZoomtofit2()

'Extract information about the parabola
pApex = SkParabola.GetApexPoint2
pStart = SkParabola.GetStartPoint2
pEnd = SkParabola.GetEndPoint2
pFocal = SkParabola.GetFocalPoint2

Debug.Print("      Apex Point = (" & pApex.X * 1000 & ", " & pApex.Y * 1000 & ", " & pApex.Z * 1000 & ") mm")
Debug.Print("      Start Point = (" & pStart.X * 1000 & ", " & pStart.Y * 1000 & ", " & pStart.Z * 1000 & ") mm")
Debug.Print("      End Point = (" & pEnd.X * 1000 & ", " & pEnd.Y * 1000 & ", " & pEnd.Z * 1000 & ") mm")
Debug.Print("      Focal Point = (" & pFocal.X * 1000 & ", " & pFocal.Y * 1000 & ", " & pFocal.Z * 1000 & ") mm")

' Define point parameters
If Not pFocal.Z = 0 Or Not pApex.Z = 0 Or Not pStart.Z = 0 Or Not pEnd.Z = 0 Then
    MsgBox("Need a 2D sketch.")
    Exit Sub
End If

' Algebraic equation of parabola
Dim h As Double
Dim p As Double
Dim k As Double

h = pApex.X
k = pApex.Y

' Correct anomalies when the parabola is aligned with the x and y axes
If pFocal.Y = pApex.Y Then
    If pFocal.X > pApex.X Then
        p = Sqrt((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2)
    Else
        p = -(Sqrt((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2))
    End If
End If

If pFocal.X = pApex.X Then
    If pFocal.Y > pApex.Y Then
        p = Sqrt((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2)
    Else
        p = -(Sqrt((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2))
    End If
End If

If pFocal.X <> pApex.X And pFocal.Y <> pApex.Y Then
    p = (Sqrt((pFocal.Y - pApex.Y) ^ 2 + (pFocal.X - pApex.X) ^ 2))
End If

Dim a As Double
Dim b As Double
Dim c As Double
Dim c1 As Double
Dim c2 As Double
Dim c3 As Double
Dim c4 As Double
Dim c5 As Double
Dim c6 As Double
Dim theta As Double

' Obtain the correct value for theta as the parabola rotates
If pFocal.X <> pApex.X And pFocal.Y <> pApex.Y Then
    a = 1 / (4 * p)
    b = -k / (2 * p)
    c = (k * k / (4 * p)) + h

    ' Theta in first quadrant
    If pFocal.Y > pApex.Y And pFocal.X > pApex.X Then
        theta = Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))
    End If

    ' Theta in second quadrant
    If pFocal.Y > pApex.Y And pFocal.X < pApex.X Then
        theta = (Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + pi
    End If

    ' Theta in the third quadrant
    If pFocal.Y < pApex.Y And pFocal.X < pApex.X Then
        theta = (Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + pi
    End If

    ' Theta in the fourth quadrant
    If pFocal.Y < pApex.Y And pFocal.X > pApex.X Then
        theta = (Atan((pFocal.Y - pApex.Y) / (pFocal.X - pApex.X))) + (2 * pi)
    End If

    c1 = Round(a * (Sin(theta)) ^ 2, 2)
    c2 = Round(-a * Sin(2 * theta), 2)
    c3 = Round(a * (Cos(theta)) ^ 2, 2)
    c4 = Round((-b * Sin(theta)) - Cos(theta), 2)
    c5 = Round((b * Cos(theta)) - Sin(theta), 2)
    c6 = Round(c, 2)

```

```

        Debug.Print("Equation of the parabola: " & c1 & "x^2 + " & c2 & "xy + " & c3 & "y^2 + " & c4 & "x + " & c5 & "y + " &
c6 & " = 0")

    End If

    ' If the parabola has a vertical axis of symmetry
    If pFocal.X = pApex.X Then
        a = 1 / (4 * p)
        b = -h / (2 * p)
        c = (h ^ 2 / (4 * p)) + k
        c1 = Round(a, 2)
        c4 = Round(b, 2)
        c6 = Round(c, 2)
        Debug.Print("Equation of the parabola: y = " & c1 & "x^2 + " & c4 & "x + " & c6)
    End If

    ' If the parabola has a horizontal axis of symmetry
    If pFocal.Y = pApex.Y Then
        a = 1 / (4 * p)
        b = -k / (2 * p)
        c = (k * k / (4 * p)) + h
        c3 = Round(a, 2)
        c5 = Round(b, 2)
        c6 = Round(c, 2)
        Debug.Print("Equation of the parabola: x =" & c3 & "y^2 + " & c5 & "y + " & c6)
    End If

    swSkMgr.InsertSketch(True)

End Sub

Public Function SelectPlane(ByVal Plane As ModelDoc2) As Boolean

    Dim featureTemp As Feature
    featureTemp = Plane.FirstFeature

    While Not featureTemp Is Nothing
        Dim sFeatureName As String
        sFeatureName = featureTemp.GetTypeName2

        If sFeatureName = "RefPlane" Then
            featureTemp.Select2(True, 0)
            SelectPlane = True
            Exit Function
        End If

        featureTemp = featureTemp.GetNextFeature
    End While

End Function

Public swApp As SldWorks

End Class

```

Create Partial Chamfer Example (C#)

This example shows how to create a partial chamfer.

```
// =====
// Preconditions:
// 1. Open Public_documents\samples\tutorial\api\Block20.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates Chamfer1.
// 2. Chamfer1 contains two partial chamfers.
// 3. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes to it.
// =====
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
```

```
namespace CreatePartialChamfer_CSharp
```

```
{

    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private ModelDocExtension swModelDocExt;
        private FeatureManager swFeatMgr;
        private SelectionMgr swSelMgr;
        private Feature swFeat;
        private SimpleFilletFeatureData2 swFilletData;
        private PartialEdgeFilletData PartialEdgeAFilletData;
        private PartialEdgeFilletData PartialEdgeBFilletData;
        private Edge filletItemA;
        private Edge filletItemB;
        private object swFeatDataObj;

        private bool boolstatus;
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swFeatMgr = swModel.FeatureManager;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Create the fillet feature data object to be a simple fillet type
            swFeatDataObj = swFeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmFillet);
            swFilletData = (SimpleFilletFeatureData2)swFeatDataObj;

            // Initialize the feature type to be a constant radius fillet
            swFilletData.Initialize((int)swSimpleFilletType_e.swConstRadiusFillet);

            // Specialize the feature type to be a chamfer
            swFilletData.ConicTypeForCrossSectionProfile = (int)swFeatureFilletProfileType_e.swFeatureFilletConicRhoZeroChamfer;

            swFilletData.OverflowType = (int)swFilletOverflowType_e.swFilletOverflowType_Default;
            swFilletData.DefaultRadius = 0.01;

            // Select two edges to partially chamfer
            boolstatus = swModel.Extension.SelectByRay(-0.0627751435901871, 0.0395396450382464, 0.000603865270420556,
            0.156569748366562, -0.551708952719479, -0.81920885333693, 0.0011220164860728, 1, false, 1, 0);
```

```
boolstatus = swModel.Extension.SelectByRay(0.0644200432014657, 0.039572847309671, 0.0130368065144353,
0.156569748366562, -0.551708952719479, -0.81920885333693, 0.0011220164860728, 1, true, 1, 0);
```

```
    filletItemA = (Edge)swSelMgr.GetSelectedObject6(1, -1);
    filletItemB = (Edge)swSelMgr.GetSelectedObject6(2, -1);
    int errorCode;
    swFilletData.EnablePartialEdgeParameters = true;
    PartialEdgeAFilletData = (PartialEdgeFilletData)swFilletData.GetPartialEdgeFilletData(filletItemA);
    errorCode = PartialEdgeAFilletData.SetPartialFilletParameters(true,
(int)swSimpleFilletPartialEdgeCondition_e.swPartialEdgeDistanceOffset, 0.01, null,
(int)swSimpleFilletPartialEdgeCondition_e.swPartialEdgeDistanceOffset, 0.03, null);
    PartialEdgeBFilletData = (PartialEdgeFilletData)swFilletData.GetPartialEdgeFilletData(filletItemB);
    errorCode = PartialEdgeBFilletData.SetPartialFilletParameters(true,
(int)swSimpleFilletPartialEdgeCondition_e.swPartialEdgeDistanceOffset, 0.01, null,
(int)swSimpleFilletPartialEdgeCondition_e.swPartialEdgeDistanceOffset, 0.03, null);
    bool warning;
    swFeat = swFeatMgr.CreateFeature(swFilletData);
    errorCode = swFeat.GetErrorcode2(out warning);
}

/// <summary>
///     ''' The SldWorks swApp variable is pre-assigned for you.
///     ''' </summary>
public SldWorks swApp;
}
```

```
}
```

Create Path Length Dimension Example (C#)

This example shows how to create a path length dimension.

```
//-----
// Preconditions: Open a new part document.
//
// Postconditions: Select Sketch1 in the FeatureManager design tree to
// see the path length dimension of 15.75.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace CreatePathLengthDim_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        DisplayDimension myDisplayDim;
        bool boolstatus;

        public void Main()
        {
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swInputDimValOnCreate, false);

            Part = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.075, 0.05, 0, 0.05, -0.025, 0);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.SketchManager.MakeSketchChain();
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0.00661546453402301, 0.0508003665223665, 0, false, 0,
null, 0);

            myDisplayDim = (DisplayDimension)Part.Extension.AddPathLengthDim(-0.0580395474035344, 0.0841706952643316, 0);
            Part.SketchManager.InsertSketch(true);
        }

        public SldWorks swApp;
    }
}
```

Create Path Length Dimension Example (VBA)

This example shows how to create a path length dimension.

' Preconditions: Open a new part document.
,

' Postconditions: Select Sketch1 in the FeatureManager design tree to see
' the path length dimension of 15.75.

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myDisplayDim As SldWorks.DisplayDimension

Dim boolstatus As Boolean
Option Explicit
Sub main()
    Set swApp = Application.SldWorks

    swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swInputDimValOnCreate, False

    Set Part = swApp.ActiveDoc
    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Part.SketchManager.InsertSketch True
    Part.ClearSelection2 True
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-0.075, 0.05, 0, 0.05, -0.025, 0)
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = Part.SketchManager.MakeSketchChain()
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 6.61546453402301E-03, 5.08003665223665E-02, 0, False, 0, Nothing, 0)

    Set myDisplayDim = Part.Extension.AddPathLengthDim(-5.80395474035344E-02, 8.41706952643316E-02, 0)
    Part.SketchManager.InsertSketch True

End Sub
```

Create Path Length Dimension Example (VB.NET)

This example shows how to create a path length dimension.

```
'-----
' Preconditions: Open a new part document.
'
' Postconditions: Select Sketch1 in the FeatureManager design tree to
' see the path length dimension of 15.75.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
    Dim myDisplayDim As DisplayDimension
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swInputDimValOnCreate, False)
```

```
        Part = swApp.ActiveDoc
```

```
        boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Part.SketchManager.InsertSketch(True)
```

```
        Part.ClearSelection2(True)
```

```
        Dim vSkLines As Object
```

```
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.075, 0.05, 0, 0.05, -0.025, 0)
```

```
        Part.ClearSelection2(True)
```

```
        boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
```

```
        boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
```

```
        boolstatus = Part.SketchManager.MakeSketchChain()
```

```
        Part.ClearSelection2(True)
```

```
        boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0.00661546453402301, 0.0508003665223665, 0, False, 0,
Nothing, 0)
```

```
        myDisplayDim = Part.Extension.AddPathLengthDim(-0.0580395474035344, 0.0841706952643316, 0)
```

```
        Part.SketchManager.InsertSketch(True)
```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Path Segment Structure System Member Example (C#)

This example shows how to create structure system path segment members that are split along a reference plane.

```
// =====
// Preconditions:
// 1. Ensure the specified part template exists.
// 2. Open an Immediate window.
// 3. Press F5.
//
// Postconditions:
// 1. Creates Sketch1 containing two sketch segments.
// 2. Creates Plane1.
// 3. Configures the start/end extensions and the member profile.
// 4. Selects two sketch segments and Plane1.
// 5. Configures structure system to split members using Plane1.
// 6. Creates a Structure System1 with four split members: Member1_1, Member1_2, Member2_1, and Member2_2.
// 7. Inspect the structure system in the graphics area.
// 8. Press F5 to edit the structure system.
// 9. Changes the start/end extensions for Member1_1 and Member1_2.
// 10. Modifies the structure system.
// 11. Inspect the changes in the graphics area.
// =====
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

```
namespace PathSegment_CSharp
```

```
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 modDoc;
        private FeatureManager swFeatMgr;
        private SelectionMgr swSelMgr;
        private ModelDocExtension modDocExt;
        private StructureSystemMemberFeatureData structMemDef;
        private StructureSystemMemberProfile profDef;
        private PrimaryStructuralMemberFeatureData PrimDef;
        private PrimaryMemberPathSegmentFeatureData memPathSegDef;
        private object[] segments = new object[2];
        private bool stat;
        private StructureSystemMemberFeatureData[] memData = new StructureSystemMemberFeatureData[1];
        private object[] memDataArray;
        private Feature structSys;
        private Feature feat;
        private StructureSystemFolder structSysModDef;
        private object[] outProfiles;
        private StructureSystemMemberFeatureData MemberData;
        private object[] memberArray;
        private Feature profileGrpFeat;
        private ProfileGroupFolder profileGrp;
        private StructureSystemMemberFeatureData memToChange;
        private int i;
        private int j;

        public DispatcherWrapper[] ObjectArrayToDispatcherWrapperArray(object[] Objects)
        {

```

```

        int ArraySize;
        ArraySize = Objects.GetUpperBound(0);
        var d = new DispatcherWrapper[ArraySize + 1];
        int ArrayIndex;
        var loopTo = ArraySize;
        for (ArrayIndex = 0; ArrayIndex <= loopTo; ArrayIndex++)
            d[ArrayIndex] = new DispatcherWrapper(Objects[ArrayIndex]);
        return d;
    }

    public void Main()
    {
        double swSheetWidth;
        swSheetWidth = 0d;
        double swSheetHeight;
        swSheetHeight = 0d;
        modDoc = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0,
        swSheetWidth, swSheetHeight);
        modDoc = (ModelDoc2)swApp.ActiveDoc;
        modDocExt = modDoc.Extension;
        swSelMgr = (SelectionMgr)modDoc.SelectionManager;
        modDoc.SketchManager.InsertSketch(true);
        stat = modDocExt.SelectByID2("Front Plane", "PLANE", -0.0550812963393624d, 0.0228249965575248d, 0.0135586835718293d,
        false, 0, null, 0);
        modDoc.ClearSelection2(true);
        object skSegment;
        skSegment = modDoc.SketchManager.CreateLine(-0.036101d, -0.017052d, 0d, 0.03111d, 0.018383d, 0d);
        skSegment = modDoc.SketchManager.CreateLine(0.055067d, -0.015222d, 0d, -0.01281d, -0.055483d, 0d);
        modDoc.SketchManager.InsertSketch(true);
        modDoc.ClearSelection2(true);
        stat = modDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
        RefPlane myRefPlane;
        myRefPlane = (RefPlane)modDoc.FeatureManager.InsertRefPlane(8, 0.02d, 0, 0, 0, 0);
        modDoc.ClearSelection2(true);
        stat = modDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.0123094293114227d, -0.00450860223252535d, 0, true,
        0, null, 0);
        stat = modDocExt.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", 1.11728134432853d, -0.0376344266391984d, 0, true, 0,
        null, 0);
        stat = modDocExt.SelectByID2("Plane1", "PLANE", 0.0199883401957978d, 0.0202450835544425d, -0.0243017856732353d, true,
        0, null, 0);
        structMemDef = (StructureSystemMemberFeatureData)modDocExt.CreateStructureSystemMemberData(0);
        Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " +
        structMemDef.StructureSystemMemberType);
        structMemDef.StartEndExtendD1 = 0.1d;
        Debug.Print("Extend length in direction 1 (m): " + structMemDef.StartEndExtendD1);
        structMemDef.StartEndExtendD2 = 0.2d;
        Debug.Print("Extend length in direction 2 (m): " + structMemDef.StartEndExtendD2);
        profDef = (StructureSystemMemberProfile)structMemDef.MemberProfile;
        profDef.ProfileStandard = "ansi inch";
        profDef.ProfileType = "c channel";
        profDef.ProfileSize = "3 x 5";
        PrimDef = (PrimaryStructuralMemberFeatureData)structMemDef;
        Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " +
        PrimDef.PrimaryMemberType);
        memPathSegDef = (PrimaryMemberPathSegmentFeatureData)PrimDef;
        memPathSegDef.MergeTangentMembers = true;
        Debug.Print("Merge tangent members? " + memPathSegDef.MergeTangentMembers);
        segments[0] = swSelMgr.GetSelectedObject6(1, 0);
        segments[1] = swSelMgr.GetSelectedObject6(2, 0);
        DispatcherWrapper[] dArray;
        dArray = ObjectArrayToDispatcherWrapperArray(segments);
        stat = memPathSegDef.SetPathSegments(dArray);
        Debug.Print("Path segments set successfully: " + stat);
        var planes = new object[1];
        planes[0] = swSelMgr.GetSelectedObject6(3, 0);
    }
}

```

```

structMemDef.IsSplit = true;
StructureSystemSplitMember structSplitMember;
structSplitMember = (StructureSystemSplitMember)structMemDef.GetSplitMember();
structSplitMember.MemberType = (int)swStructureSplitMemberType_e.swStructureSplitMember_Reference;
DispatchWrapper[] eArray;
eArray = ObjectArrayToDispatchWrapperArray(planes);
stat = structSplitMember.SetSplitReferences(eArray);
memData[0] = structMemDef;
DispatchWrapper[] fArray;
fArray = ObjectArrayToDispatchWrapperArray(memData);
structSys = modDocExt.CreateStructureSystem(fArray, null);
modDoc.ViewZoomtofit();
Debugger.Break();

// Edit the structure system
structSysModDef = (StructureSystemFolder)structSys.GetSpecificFeature2();
Debug.Print("Number of profile group folders: " + structSysModDef.GetProfileGroupFoldersCount());
outProfiles = (object[])structSysModDef.GetProfileGroupFolders();
if (outProfiles.Length > 0)
{
    var loopTo = outProfiles.GetUpperBound(0);
    for (I = 0; I <= loopTo; I++)
    {
        profileGrpFeat = (Feature)outProfiles[I];
        profileGrp = (ProfileGroupFolder)profileGrpFeat.GetSpecificFeature2();
        if (profileGrp is object)
        {
            Debug.Print("Number of members inside profile group folder: " +
profileGrp.GetStructureSystemMemberCount());
            memberArray = (object[])profileGrp.GetStructureSystemMembers();
            if (memberArray.Length > 0)
            {
                var loopTo1 = memberArray.GetUpperBound(0);
                for (j = 0; j <= loopTo1; j++)
                {
                    MemberData = (StructureSystemMemberFeatureData)memberArray[j]; // Get Member1 only
                    break;
                }
            }
        }
    }
}

memTochange = MemberData;

// Change the start/end extensions for Member1
Debug.Print("Change member extensions in directions 1 and 2 for one member only");
feat = memTochange.GetFeature();
memTochange.StartEndExtendD1 = 0.5d;
memTochange.StartEndExtendD2 = 0.2d;

// Save edits
stat = feat.ModifyDefinition(MemberData, modDoc, null);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Create Path Segment Structure System Member Example (VB.NET)

This example shows how to create structure system path segment members that are split along a reference plane.

```
'=====
'Preconditions:
'1. Ensure the specified part template exists.
'2. Open an Immediate window.
'3. Press F5.
'
'Postconditions:
' 1. Creates Sketch1 containing two sketch segments.
' 2. Creates Plane1.
' 3. Configures the start/end extensions and the member profile.
' 4. Selects two sketch segments and Plane1.
' 5. Configures structure system to split members using Plane1.
' 6. Creates a Structure System1 with four split members: Member1_1, Member1_2, Member2_1, and Member2_2.
' 7. Inspect the structure system in the graphics area.
' 8. Press F5 to edit the structure system.
' 9. Changes the start/end extensions for Member1_1 and Member1_2.
'10. Modifies the structure system.
'11. Inspect the changes in the graphics area.
'=====

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

Partial Class SolidWorksMacro

```
    Dim modDoc As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim swSelMgr As SelectionMgr
    Dim modDocExt As ModelDocExtension
    Dim structMemDef As StructureSystemMemberFeatureData
    Dim profDef As StructureSystemMemberProfile
    Dim PrimDef As PrimaryStructuralMemberFeatureData
    Dim memPathSegDef As PrimaryMemberPathSegmentFeatureData
    Dim segments(1) As Object
    Dim stat As Boolean
    Dim memData(0) As StructureSystemMemberFeatureData
    Dim memDataArray() As Object
    Dim structSys As Feature
    Dim feat As Feature
    Dim structSysModDef As StructureSystemFolder
    Dim outProfiles() As Object
    Dim MemberData As StructureSystemMemberFeatureData
    Dim memberArray() As Object
    Dim profileGrpFeat As Feature
    Dim profileGrp As ProfileGroupFolder
    Dim memTochange As StructureSystemMemberFeatureData
    Dim i As Integer
    Dim j As Integer

    Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
        Dim ArraySize As Integer
        ArraySize = Objects.GetUpperBound(0)
        Dim d(ArraySize) As DispatchWrapper
```

```

Dim ArrayIndex As Integer
For ArrayIndex = 0 To ArraySize
    d(ArrayIndex) = New DispatcherWrapper(Objects(ArrayIndex))
Next
Return d
End Function

Sub main()

    Dim swSheetWidth As Double
    swSheetWidth = 0
    Dim swSheetHeight As Double
    swSheetHeight = 0
    modDoc = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.PRTDOT", 0, swSheetWidth,
swSheetHeight)
    modDoc = swApp.ActiveDoc

    modDocExt = modDoc.Extension
    swSelMgr = modDoc.SelectionManager

    modDoc.SketchManager.InsertSketch(True)
    stat = modDocExt.SelectByID2("Front Plane", "PLANE", -0.0550812963393624, 0.0228249965575248, 0.0135586835718293, False, 0,
Nothing, 0)
    modDoc.ClearSelection2(True)
    Dim skSegment As Object
    skSegment = modDoc.SketchManager.CreateLine(-0.036101, -0.017052, 0#, 0.03111, 0.018383, 0#)
    skSegment = modDoc.SketchManager.CreateLine(0.055067, -0.015222, 0#, -0.01281, -0.055483, 0#)

    modDoc.SketchManager.InsertSketch(True)
    modDoc.ClearSelection2(True)

    stat = modDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Dim myRefPlane As RefPlane
    myRefPlane = modDoc.FeatureManager.InsertRefPlane(8, 0.02, 0, 0, 0, 0)
    modDoc.ClearSelection2(True)

    stat = modDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.0123094293114227, -0.00450860223252535, 0, True, 0,
Nothing, 0)
    stat = modDocExt.SelectByID2("Line2@Sketch1", "EXTSKETCHSEGMENT", 1.11728134432853, -0.0376344266391984, 0, True, 0,
Nothing, 0)
    stat = modDocExt.SelectByID2("Plane1", "PLANE", 0.0199883401957978, 0.0202450835544425, -0.0243017856732353, True, 0,
Nothing, 0)

    structMemDef = modDocExt.CreateStructureSystemMemberData(0)

    Debug.Print("Type of structure system member as defined by swStructureSystemMemberType_e: " &
structMemDef.StructureSystemMemberType)

    structMemDef.StartEndExtendD1 = 0.1
    Debug.Print("Extend length in direction 1 (m): " & structMemDef.StartEndExtendD1)
    structMemDef.StartEndExtendD2 = 0.2
    Debug.Print("Extend length in direction 2 (m): " & structMemDef.StartEndExtendD2)

    profDef = structMemDef.MemberProfile

    profDef.ProfileStandard = "ansi inch"
    profDef.ProfileType = "c channel"
    profDef.ProfileSize = "3 x 5"

    PrimDef = structMemDef
    Debug.Print("Structure system primary member creation type as defined by swStructureSystemMemberCreationType_e: " &
PrimDef.PrimaryMemberType)

    memPathSegDef = PrimDef

```

```

memPathSegDef.MergeTangentMembers = True
Debug.Print("Merge tangent members? " & memPathSegDef.MergeTangentMembers)

segments(0) = swSelMgr.GetSelectedObject6(1, 0)
segments(1) = swSelMgr.GetSelectedObject6(2, 0)

Dim dArray() As DispatcherWrapper
dArray = ObjectArrayToDispatcherWrapperArray(segments)

stat = memPathSegDef.SetPathSegments(dArray)
Debug.Print("Path segments set successfully: " & stat)

Dim planes(0) As Object
planes(0) = swSelMgr.GetSelectedObject6(3, 0)
structMemDef.IsSplit = True
Dim structSplitMember As StructureSystemSplitMember
structSplitMember = structMemDef.GetSplitMember
structSplitMember.MemberType = swStructureSplitMemberType_e.swStructureSplitMember_Reference

Dim eArray() As DispatcherWrapper
eArray = ObjectArrayToDispatcherWrapperArray(planes)

stat = structSplitMember.SetSplitReferences(eArray)

memData(0) = structMemDef

Dim fArray() As DispatcherWrapper
fArray = ObjectArrayToDispatcherWrapperArray(memData)

structSys = modDocExt.CreateStructureSystem(fArray, Nothing)

modDoc.ViewZoomtofit

Stop

'Edit the structure system
structSysModDef = structSys.GetSpecificFeature2
Debug.Print("Number of profile group folders: " & structSysModDef.GetProfileGroupFoldersCount)

outProfiles = structSysModDef.GetProfileGroupFolders()

If (outProfiles.Length > 0) Then
    For I = 0 To UBound(outProfiles)
        profileGrpFeat = outProfiles(I)
        profileGrp = profileGrpFeat.GetSpecificFeature2
        If Not profileGrp Is Nothing Then
            Debug.Print("Number of members inside profile group folder: " & profileGrp.GetStructureSystemMemberCount)
            memberArray = profileGrp.GetStructureSystemMembers()
            If (memberArray.Length > 0) Then
                For j = 0 To UBound(memberArray)
                    MemberData = memberArray(j) 'Get Member1 only
                Exit For
            Next
        End If
    Next
End If

memToChange = MemberData

```

```
'Change the start/end extensions for Member1
Debug.Print("Change member extensions in directions 1 and 2 for one member only")
feat = memTochange.GetFeature()
memTochange.StartEndExtendD1 = 0.5
memTochange.StartEndExtendD2 = 0.2

'Save edits
stat = feat.ModifyDefinition(MemberData, modDoc, Nothing)
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class
```

Create Planar Surface Feature Example (C#)

This example shows how to create a planar surface feature.

```
//-----
// Preconditions:
// 1. Verify that the specified document template exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a new model document with a boss extrusion.
// 2. Creates Surface-Radiate1 and Surface-Plane1 in the FeatureManager
//    design tree.
// 3. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateSurfacePlaneFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SelectionMgr swSelMgr;
        SelectData swSelData;
        SurfaceRadiateFeatureData swRadiate;
        SurfacePlanarFeatureData swPlane;
        Feature swFeat;
        Entity swEnt;
        object[] vRadEnt;
        object[] vBoundEnt;
        Entity swRadDirEnt;
        int i;

        bool boolStatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2015\\templates\\Part.prt", 0, 0, 0);
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolStatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.0448901407839529, 0.0279954694016864, 0.0046682067411,
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            boolStatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swl
            boolStatus = Part.Extension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int
            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0555749908365768, 0.0329075527136081, 0, 0.0478203409524033, -0.0
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
            Part.ShowNamedView2("Trimetric", 8);
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            boolStatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);

            object myFeature = null;
            myFeature = Part.FeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
            0, 0, false);

            boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0447337592343047, 0.0328467250718631, 0.00258132540182032, false,
            boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0556265649287866, 0.0156695101210289, 0.0025672149453726, true,
            boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0140113588298618, -0.0317157034173761, 0.00254079743683633, true,
            boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047780958393389, -0.00542256709667299, 0.00256078163948814, true,
            boolStatus = Part.Extension.SelectByID2("", "FACE", 0.0478203409524554, -0.00305747564971171, 0.000546558985774936, true

            Part.InsertRadiateSurface(0.0254, false, false);

            swSelMgr = (SelectionMgr)Part.SelectionManager;
            swSelData = swSelMgr.CreateSelectData();

            boolStatus = Part.Extension.SelectByID2("Surface-Radiate1", "REFSURFACE", 0, 0, 0, false, 0, null, 0);

            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swRadiate = (SurfaceRadiateFeatureData)swFeat.GetDefinition();

            // Get radiate surface data
            Debug.Print("File = " + Part.GetPathName());
            Debug.Print(" " + swFeat.Name);
            Debug.Print(" Distance: " + swRadiate.Distance * 1000.0 + " mm");
            Debug.Print(" Flip? " + swRadiate.Flip);
        }
    }
}
```

```

Debug.Print("    Propagate to tangent faces? " + swRadiate.PropagateToTangentFaces);

// Roll back to get direction reference and radiated edges
boolStatus = swRadiate.AccessSelections(Part, null);
swRadDirEnt = (Entity)swRadiate.DirectionReference;
Part.ClearSelection2(true);

vRadEnt = (object[])swRadiate.RadiatedEntities;

Debug.Print("Type as defined in swSelectType_e:");
for (i = 0; i <= swRadiate.GetRadiatedEntitiesCount() - 1; i++)
{
    swEnt = (Entity)vRadEnt[i];
    Debug.Print("    Radiated Entity(" + i + ") = " + swEnt.GetType());
}

swRadiate.ReleaseSelectionAccess();

Part.ClearSelection2(true);

boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0150796523155918, 0.0330635737250304, 0.0277193529692568, false, 1, null, 0);
boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.0556924155575302, -0.00146678127401856, 0.0280574247206573, true, 1, null, 0);
boolStatus = Part.Extension.SelectByID2("", "EDGE", -0.00737846281163002, -0.0317262656324715, 0.0279565971788429, true, 1, null, 0);
boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047950275020753, 0.00971418585908168, 0.0278100659315328, true, 1, null, 0);

boolStatus = Part.InsertPlanarRefSurface();
Part.ClearSelection2(true);

boolStatus = Part.Extension.SelectByID2("Surface-Plane1", "REFSURFACE", 0, 0, 0, false, 0, null, 0);

// Roll back to get the planar surface bounding edges
swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
swPlane = (SurfacePlanarFeatureData)swFeat.GetDefinition();

boolStatus = swPlane.AccessSelections(Part, null);
vBoundEnt = (object[])swPlane.BoundingEntities;

Debug.Print("Types of planar surface bounding entities as defined in swSelectType_e:");

for (i = 0; i <= swPlane.GetBoundingEntitiesCount() - 1; i++) {
    swEnt = (Entity)vBoundEnt[i];
    Debug.Print("    Bounding entity(" + i + ") = " + swEnt.GetType());
}

swPlane.ReleaseSelectionAccess();

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Create Planar Surface Feature Example (VBA)

This example shows how to create a planar surface feature.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a new model document with a boss extrusion.
' 2. Creates Surface-Radiate1 and Surface-Plane1 in the FeatureManager
'    design tree.
' 3. Inspect the Immediate window.
'-----

Dim swApp          As SldWorks.SldWorks
Dim Part           As SldWorks.ModelDoc2
Dim swSelMgr       As SldWorks.SelectionMgr
Dim swSelData      As SldWorks.SelectData
Dim swRadiate      As SldWorks.SurfaceRadiateFeatureData
Dim swPlane        As SldWorks.SurfacePlanarFeatureData
Dim swFeat         As SldWorks.Feature
Dim swEnt          As SldWorks.Entity
Dim vRadEnt        As Variant
Dim vBoundEnt      As Variant
Dim swRadDirEnt    As SldWorks.Entity
Dim i              As Long
Dim boolStatus     As Boolean
Option Explicit

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prt", 0, 0, 0)
    Set Part = swApp.ActiveDoc

    boolStatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -4.48901407839529E-02, 2.79954694016864E-02, 4.66820674117181E-03, False, 0, Nothing, 0)
    Part.SketchManager.InsertSketch True
    Part.ClearSelection2 True
    boolStatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    boolStatus = Part.Extension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-5.55749908365768E-02, 3.29075527136081E-02, 0, 4.78203409524033E-02, -3.17145296545045E-02, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    Part.ShowNamedView2 "Trimetric", 8
    Part.SketchManager.InsertSketch True
    Part.ClearSelection2 True
    boolStatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

    Dim myFeature As Object
    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)

    boolStatus = Part.Extension.SelectByID2("", "EDGE", -4.47337592343047E-02, 3.28467250718631E-02, 2.58132540182032E-03, False, 2, Nothing, 0)
    boolStatus = Part.Extension.SelectByID2("", "EDGE", -5.56265649287866E-02, 1.56695101210289E-02, 2.5672149453726E-03, True, 2, Nothing, 0)
    boolStatus = Part.Extension.SelectByID2("", "EDGE", -1.40113588298618E-02, -3.17157034173761E-02, 2.54079743683633E-03, True, 2, Nothing, 0)
    boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047780958393389, -5.42256709667299E-03, 2.56078163948814E-03, True, 2, Nothing, 0)
    boolStatus = Part.Extension.SelectByID2("", "FACE", 4.78203409524554E-02, -3.05747564971171E-03, 5.46558985774936E-04, True, 1, Nothing, 0)

    Part.InsertRadiateSurface 0.0254, False, False

    Set swSelMgr = Part.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    boolStatus = Part.Extension.SelectByID2("Surface-Radiate1", "REFSURFACE", 0, 0, 0, False, 0, Nothing, 0)

    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swRadiate = swFeat.GetDefinition

    ' Get radiate surface data
    Debug.Print "File = " & Part.GetPathName
    Debug.Print " " & swFeat.Name
    Debug.Print " Distance: " & swRadiate.Distance * 1000# & " mm"
    Debug.Print " Flip? " & swRadiate.Flip
    Debug.Print " Propagate to tangent faces? " & swRadiate.PropagateToTangentFaces

    ' Roll back to get direction reference and radiated edges
    boolStatus = swRadiate.AccessSelections(Part, Nothing)

```

```

Set swRadDirEnt = swRadiate.DirectionReference
Part.ClearSelection2 True
vRadEnt = swRadiate.RadiatedEntities
Debug.Print "Type of radiated entities as defined in swSelectType_e:"
For i = 0 To swRadiate.GetRadiatedEntitiesCount - 1
    Set swEnt = vRadEnt(i)
    Debug.Print "    Radiated entity(" & i & ") = " & swEnt.GetType
Next i
swRadiate.ReleaseSelectionAccess

Part.ClearSelection2 True
boolStatus = Part.Extension.SelectByID2("", "EDGE", -1.50796523155918E-02, 3.30635737250304E-02, 2.77193529692568E-02, False,
1, Nothing, 0)
boolStatus = Part.Extension.SelectByID2("", "EDGE", -5.56924155575302E-02, -1.46678127401856E-03, 2.80574247206573E-02, True,
1, Nothing, 0)
boolStatus = Part.Extension.SelectByID2("", "EDGE", -7.37846281163002E-03, -3.17262656324715E-02, 2.79565971788429E-02, True,
1, Nothing, 0)
boolStatus = Part.Extension.SelectByID2("", "EDGE", 0.047950275020753, 9.71418585908168E-03, 2.78100659315328E-02, True, 1,
Nothing, 0)

boolStatus = Part.InsertPlanarRefSurface()
Part.ClearSelection2 True

boolStatus = Part.Extension.SelectByID2("Surface-Plane1", "REFSURFACE", 0, 0, 0, False, 0, Nothing, 0)

' Roll back to get the planar surface bounding edges
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swPlane = swFeat.GetDefinition
boolStatus = swPlane.AccessSelections(Part, Nothing)
vBoundEnt = swPlane.BoundingEntities

Debug.Print "Types of planar surface bounding entities as defined in swSelectType_e:"
For i = 0 To swPlane.GetBoundingEntitiesCount - 1
    Set swEnt = vBoundEnt(i)
    Debug.Print "    Bounding entity(" & i & ") = " & swEnt.GetType
Next i
swPlane.ReleaseSelectionAccess

End Sub

```